Name: Hafiza Tehreem Fatima

Registration: 2023-bs-ai-026

Submitted to: Mr. Samraiz

Department: Artificial Intelligence

Contents

LAB - 【01】

What is Python?

Python is a **high-level programming language** used to tell a computer what to do. It is **easy to read**, **easy to write**, and **very popular** in fields like:

- Artificial Intelligence
- Machine Learning
- Data Science
- Web Development
- Automation

Python programs are written using **simple English-like words**, which makes it beginner-friendly.

Why Python is Popular

- Easy syntax (looks close to English)
- No need to compile (runs directly)
- Large number of libraries (NumPy, Pandas, Scikit-learn)
- Used by Google, Netflix, Instagram, etc.

Basic Structure of a Python Program

```
print("Hello, Python")
```

- print() → built-in function to display output
- "Hello, Python" → text (string)

Variables

A variable is used to **store data**.

Data Types (Basic)

Type	Example
int	10, -5
float	3.14, 85.5
string	"Python"
boolean	True, False

Operators

Arithmetic Operators

```
a = 10
b = 3
print(a + b)   # Addition
print(a - b)   # Subtraction
print(a * b)   # Multiplication
print(a / b)   # Division
```

Conditional Statements (if-else)

```
marks = 75

if marks >= 50:
    print("Pass")
else:
    print("Fail")
```

Loops

for Loop

```python
for i in range(1, 6):
    print(i)
```

while Loop

```python
i = 1
while i <= 5:
    print(i)
    i += 1
```

Lists

A list stores **multiple values**.

```python
numbers = [10, 20, 30, 40]
print(numbers[0])  # Access first element
```

Functions

A function is a block of code that **performs a task**.

```python
def add(a, b):
    return a + b

result = add(5, 3)
print(result)
```

Why Python for AI & ML

- Easy to handle data
- Strong libraries
- Less code, more work done

Lab Exercise

Question 1

[3]:
```python
# Banking Transaction Validator with PIN

# Step 1: Set a correct PIN and starting balance
correct_pin = "1234"
balance = 5000  # initial balance
attempts = 0    # wrong PIN counter

# Step 2: Allow user up to 3 attempts
while attempts < 3:
    pin = input("Enter your 4-digit PIN: ")

    if pin == correct_pin:
        print("\n✅ PIN accepted!")
        print("Your current balance is:", balance)

        # Step 3: Ask for withdrawal
        amount = float(input("Enter amount to withdraw: "))

        # Step 4: Calculate 2% service charge
        service_charge = amount * 0.02
        total_deduction = amount + service_charge

        # Step 5: Check if balance is enough
        if total_deduction <= balance:
            balance -= total_deduction
            print("\n💸 Withdrawal successful!")
            print("Service Charge (2%):", service_charge)
            print("Remaining Balance:", balance)
        else:
            print("\n❌ Insufficient balance!")
```

```
        break  # Exit after transaction

    else:
        attempts += 1
        print("❌ Invalid PIN! Attempts left:", 3 - attempts)

        if attempts == 3:
            print("\n🚫 Card Blocked due to 3 invalid attempts.")
```

```
Enter your 4-digit PIN:  1234

✅ PIN accepted!
Your current balance is: 5000
Enter amount to withdraw:  25000

❌ Insufficient balance!
```

Question 2

[*]:

```
# Grocery Store Inventory System

# Step 1: Create a dictionary of dictionaries (item: {price, stock})
inventory = {
    "apple": {"price": 100, "stock": 20},
    "banana": {"price": 40, "stock": 30},
    "milk": {"price": 150, "stock": 10},
    "bread": {"price": 80, "stock": 15}
}

cart_total = 0

# Step 2: Show available items
print("🛒 Welcome to the Grocery Store!\nAvailable items:")
for item, details in inventory.items():
    print(f"{item.title()} - Rs.{details['price']} (Stock: {details['stock']})")

# Step 3: User selection
item_name = input("\nEnter item name to buy: ").lower()
quantity = int(input("Enter quantity: "))

# Step 4: Check stock
if item_name in inventory:
    if quantity <= inventory[item_name]['stock']:
        # Calculate price
        price = inventory[item_name]['price'] * quantity
        inventory[item_name]['stock'] -= quantity
        print(f"\n✅ {quantity} {item_name}(s) added to cart. Total = Rs.{price}")

        # Apply discount slabs
        if price >= 1000:
            discount = 0.20
```

```
        # Apply discount slabs
        if price >= 1000:
            discount = 0.20
        elif price >= 500:
            discount = 0.15
        elif price >= 200:
            discount = 0.10
        else:
            discount = 0

        discount_amount = price * discount
        price_after_discount = price - discount_amount

        # Add 5% tax
        tax = price_after_discount * 0.05
        final_bill = price_after_discount + tax

        print("\n--- BILL SUMMARY ---")
        print(f"Item: {item_name.title()}")
        print(f"Total Price: Rs.{price}")
        print(f"Discount: Rs.{discount_amount}")
        print(f"Tax (5%): Rs.{tax}")
        print(f"💰 Final Bill: Rs.{final_bill}")

    else:
        print("❌ Not enough stock available.")
else:
    print("❌ Item not found in inventory.")
```

```
🛒 Welcome to the Grocery Store!
Available items:
Apple - Rs.100 (Stock: 20)
Banana - Rs.40 (Stock: 30)
Milk - Rs.150 (Stock: 10)
Bread - Rs.80 (Stock: 15)
```

```
🛒 Welcome to the Grocery Store!
Available items:
Apple - Rs.100 (Stock: 20)
Banana - Rs.40 (Stock: 30)
Milk - Rs.150 (Stock: 10)
Bread - Rs.80 (Stock: 15)

Enter item name to buy:  milk
Enter quantity:  2

✅ 2 milk(s) added to cart. Total = Rs.300

--- BILL SUMMARY ---
Item: Milk
Total Price: Rs.300
Discount: Rs.30.0
Tax (5%): Rs.13.5
💰 Final Bill: Rs.283.5
```

Question 3

[11]:
```python
# Student Grading with Subject Weightage

# Step 1: Store weightages in a tuple
weights = (0.1, 0.15, 0.25, 0.25, 0.25)  # sum = 1.0

# Step 2: Get marks for 5 subjects
marks = []
for i in range(5):
    m = float(input(f"Enter marks for subject {i+1}: "))
    marks.append(m)

# Step 3: Compute weighted average
weighted_sum = sum(m * w for m, w in zip(marks, weights))
print(f"\nWeighted Average: {weighted_sum:.2f}%")

# Step 4: Assign grade
if weighted_sum >= 90:
    grade = 'A'
elif weighted_sum >= 80:
    grade = 'B'
elif weighted_sum >= 70:
    grade = 'C'
elif weighted_sum >= 60:
    grade = 'D'
else:
    grade = 'Fail'

print("Grade:", grade)
```

```python
# Step 5: Scholarship check
if weighted_sum >= 80:
    print("🏅 Eligible for Scholarship!")
else:
    print("❌ Not eligible for Scholarship.")
```

```
Enter marks for subject 1:  80
Enter marks for subject 2:  90
Enter marks for subject 3:  85
Enter marks for subject 4:  89
Enter marks for subject 5:  95

Weighted Average: 88.75%
Grade: B
🏅 Eligible for Scholarship!
```

Question 4

[14]:
```python
# BMI + Health Recommendation

# Step 1: Input details
weight = float(input("Enter your weight (kg): "))
height = float(input("Enter your height (m): "))
age = int(input("Enter your age: "))

# Step 2: Compute BMI
bmi = weight / (height ** 2)
print(f"\nYour BMI is: {bmi:.2f}")

# Step 3: Give suggestions based on BMI and age
if bmi < 18.5:
    print("⚠️ Underweight. Eat protein-rich food and follow a light exercise plan
elif 18.5 <= bmi < 24.9:
    print("✅ Normal weight. Maintain a balanced diet and regular activity.")
elif 25 <= bmi < 29.9:
    print("⚠️ Overweight. Try brisk walking and reduce sugar intake.")
else:
    print("🚨 Obese. Consult a doctor and start a controlled diet plan.")

# Step 4: Age-based advice
if age < 18:
    print("👦 Focus on nutrition for growth.")
elif 18 <= age <= 40:
    print("💪 Maintain an active lifestyle.")
else:
    print("👴 Regular health checkups recommended.")
```

```
Enter your weight (kg):  57
Enter your height (m):  6
Enter your age:  21

Your BMI is: 1.58
⚠️ Underweight. Eat protein-rich food and follow a light exercise plan.
💪 Maintain an active lifestyle.
```

Question 5

```python
# Telecom Prepaid Recharge System

# Step 1: Plan details
plan = {
    "call_rate": 2,   # per minute
    "sms_rate": 1,    # per SMS
    "data_rate": 5    # per MB
}

# Step 2: Starting balance
initial_balance = float(input("Enter your starting balance: "))
balance = initial_balance

# Step 3: Usage input
calls = int(input("Enter call minutes used: "))
sms = int(input("Enter number of SMS sent: "))
data = int(input("Enter MBs used: "))

# Step 4: Calculate total deduction
deduction = (calls * plan["call_rate"]) + (sms * plan["sms_rate"]) + (data * plan["data_rat
balance -= deduction

print(f"\nTotal usage charges: Rs.{deduction}")
print(f"Remaining balance: Rs.{balance}")

# Step 5: Low balance warning
if balance < 0.2 * initial_balance:
    print("⚠️ Low balance! Please recharge soon.")
```

```python
# Step 5: Low balance warning
if balance < 0.2 * initial_balance:
    print("⚠️ Low balance! Please recharge soon.")

# Step 6: Bonus for large recharge
recharge = float(input("\nEnter recharge amount: "))
if recharge > 1000:
    bonus = recharge * 0.1
    recharge += bonus
    print(f"🎁 Bonus added: Rs.{bonus}")

balance += recharge
print(f"✅ Updated Balance: Rs.{balance}")
```

```
Enter your starting balance:  2500
Enter call minutes used:  2
Enter number of SMS sent:  2
Enter MBs used:  24

Total usage charges: Rs.126
Remaining balance: Rs.2374.0

Enter recharge amount:  25466
🎁 Bonus added: Rs.2546.6000000000004
✅ Updated Balance: Rs.30386.6
```

Question 6

[20]:
```python
# E-Commerce Discount Engine

# Step 1: Input details
bill_amount = float(input("Enter total bill amount: "))
membership = input("Enter membership type (Regular/Gold/Platinum): ").lower()

# Step 2: Base discount
if membership == "gold":
    discount = 0.10
elif membership == "platinum":
    discount = 0.15
else:
    discount = 0.0

# Step 3: Additional discount for large bills
if bill_amount > 10000:
    discount += 0.05

discount_amount = bill_amount * discount
price_after_discount = bill_amount - discount_amount

# Step 4: Apply tax
tax = price_after_discount * 0.05
final_amount = price_after_discount + tax
```

```
# Step 5: Print receipt
print("\n--- RECEIPT ---")
print(f"Membership: {membership.title()}")
print(f"Original Bill: Rs.{bill_amount}")
print(f"Discount Applied: Rs.{discount_amount}")
print(f"Tax (5%): Rs.{tax}")
print(f"💰 Final Amount to Pay: Rs.{final_amount}")
```

```
Enter total bill amount:  35777
Enter membership type (Regular/Gold/Platinum):  gold

--- RECEIPT ---
Membership: Gold
Original Bill: Rs.35777.0
Discount Applied: Rs.5366.550000000001
Tax (5%): Rs.1520.5225
💰 Final Amount to Pay: Rs.31930.972499999996
```

Question 7

[23]:
```
# Hospital Emergency Unit with Multi-Condition Triage

# Step 1: Get patient details
temp = float(input("Enter temperature (°C): "))
pulse = int(input("Enter pulse rate: "))
oxygen = int(input("Enter oxygen level (%): "))
bp = int(input("Enter blood pressure (mmHg): "))

# Step 2: Classify patient condition
if oxygen < 85 or bp > 180 or pulse > 130:
    print("\n🚨 Condition: CRITICAL")
    print("👉 Suggestion: Admit in ICU immediately.")
elif 85 <= oxygen < 90 or 150 < bp <= 180 or 110 < pulse <= 130:
    print("\n⚠️ Condition: URGENT")
    print("👉 Suggestion: Immediate doctor attention required.")
elif 90 <= oxygen < 95 or 130 < bp <= 150 or 90 < pulse <= 110:
    print("\n🩺 Condition: UNDER OBSERVATION")
    print("👉 Suggestion: Monitor patient closely.")
else:
    print("\n✅ Condition: STABLE")
    print("👉 Suggestion: Routine check-up and rest.")
```

```
Enter temperature (°C):  30
Enter pulse rate:  78
Enter oxygen level (%):  96
Enter blood pressure (mmHg):  116

✅ Condition: STABLE
👉 Suggestion: Routine check-up and rest.
```

Question 8

[3]:
```
total_marks = float(input("Enter total marks: "))
obtained_marks = float(input("Enter obtained marks: "))
wrong_answers = int(input("Enter number of wrong answers: "))

# Negative marking
negative_marks = wrong_answers * 0.25
final_marks = obtained_marks - negative_marks

# Prevent negative marks
if final_marks < 0:
    final_marks = 0

# Percentage calculation
percentage = (final_marks / total_marks) * 100

# Display results
print("\n--- Result ---")
print("Final Marks:", final_marks)
print("Percentage:", percentage)

# Division and scholarship logic
if percentage >= 80:
    print("Division: First Division")
    print("Scholarship: Eligible")
elif percentage >= 60:
    print("Division: Second Division")
    print("Scholarship: Not Eligible")
elif percentage >= 40:
    print("Division: Third Division")
    print("Scholarship: Not Eligible")
else:
    print("Status: Fail")
    print("Suggestion: Repeat Exam.")
```

```
Enter total marks:  89
Enter obtained marks:  89
Enter number of wrong answers:  0

--- Result ---
Final Marks: 89.0
Percentage: 100.0
Division: First Division
Scholarship: Eligible
```

Question 9

[6]:
```
age = int(input("Enter age: "))
showtime = int(input("Enter showtime (24-hour format): "))
ticket_type = input("Enter ticket type (Normal / 3D / IMAX): ").lower()

# Base prices
if ticket_type == "normal":
    price = 500
elif ticket_type == "3d":
    price = 800
elif ticket_type == "imax":
    price = 1000
else:
    print("Invalid ticket type")
    price = 0

# Child restriction
if age < 12 and showtime >= 22:
    print("Booking not allowed: Children cannot attend late-night shows.")
else:
    # Peak hour charge
    if showtime >= 18 and showtime <= 22:
        price = price + (price * 0.10)

    # Senior citizen discount
    if age >= 60:
        price = price - (price * 0.30)

    # Final output
    print("\n--- Ticket Details ---")
    print("Final Ticket Price:", price)
```

```
Enter age:  21
Enter showtime (24-hour format):  12
Enter ticket type (Normal / 3D / IMAX):  normal

--- Ticket Details ---
Final Ticket Price: 500
```

Question 10

```
temperature = float(input("Enter temperature (°C): "))
weather = input("Enter weather (Rainy / Sunny / Cold): ").lower()
event = input("Enter event type (Casual / Business / Party): ").lower()

# Outfit suggestion
if event == "business" and temperature > 35:
    outfit = "Light formal suit"
elif event == "casual":
    outfit = "Comfortable casual clothes"
elif event == "business":
    outfit = "Formal wear"
elif event == "party":
    outfit = "Stylish party outfit"
else:
    outfit = "Regular outfit"

# Accessories suggestion
accessories = []

if weather == "rainy":
    accessories.append("Umbrella")
if weather == "cold":
    accessories.append("Gloves")

# Display result
print("\n--- Suggestion ---")
print("Outfit:", outfit)

if accessories:
    print("Accessories:", ", ".join(accessories))
else:
    print("Accessories: None")
```

```
Enter temperature (°C):  32
Enter weather (Rainy / Sunny / Cold):  rainy
Enter event type (Casual / Business / Party):  party

--- Suggestion ---
Outfit: Stylish party outfit
Accessories: Umbrella
```

Question 11

[2]:
```
balance = float(input("Enter account balance: "))
amount = int(input("Enter withdrawal amount: "))

# Service charge (0.5%)
service_charge = amount * 0.005
total_deduction = amount + service_charge

# Check balance
if total_deduction > balance:
    print("Insufficient balance.")
else:
    remaining_balance = balance - total_deduction

    # Note counting
    notes_1000 = amount // 1000
    amount %= 1000

    notes_500 = amount // 500
    amount %= 500

    notes_100 = amount // 100
    amount %= 100

    notes_50 = amount // 50
    amount %= 50

    notes_10 = amount // 10
    amount %= 10

    notes_1 = amount
```

```
    notes_1 = amount

    # Receipt
    print("\n------ ATM RECEIPT ------")
    print("1000 Notes:", notes_1000)
    print("500 Notes :", notes_500)
    print("100 Notes :", notes_100)
    print("50 Notes  :", notes_50)
    print("10 Notes  :", notes_10)
    print("1 Notes   :", notes_1)
    print("-------------------------")
    print("Service Charge:", service_charge)
    print("Remaining Balance:", remaining_balance)
```

Output

```
Enter account balance:  20000
Enter withdrawal amount:  10000

------ ATM RECEIPT ------
1000 Notes: 10
500 Notes : 0
100 Notes : 0
50 Notes  : 0
10 Notes  : 0
1 Notes   : 0
-------------------------
Service Charge: 50.0
Remaining Balance: 9950.0
```

Question 12

```
days_late = int(input("Enter number of days late: "))
membership = input("Enter membership type (Normal / Premium): ").lower()

# Check membership cancellation
if days_late > 30:
    print("\nMembership Cancelled due to excessive delay.")
else:
    # Fine calculation
    if days_late <= 10:
        fine_per_day = 10
    elif days_late <= 20:
        fine_per_day = 20
    else:
        fine_per_day = 50

    total_fine = days_late * fine_per_day

    # Premium discount
    if membership == "premium":
        discount = total_fine * 0.50
    else:
        discount = 0

    final_fine = total_fine - discount

    # Fine slip
    print("\n----- Library Fine Slip -----")
    print("Days Late        :", days_late)
    print("Membership Type :", membership.capitalize())
    print("Fine per Day     :", fine_per_day)
    print("Total Fine      :", total_fine)
    print("Discount        :", discount)
    print("Final Fine      :", final_fine)
```

Output

```
Enter number of days late:  3
Enter membership type (Normal / Premium):  normal

----- Library Fine Slip -----
Days Late        : 3
Membership Type : Normal
Fine per Day     : 10
Total Fine      : 30
Discount        : 0
Final Fine      : 30
```

Question 13

```
sales = [12000, 13500, 12800, 14000, 15000, 14500,
         16000, 15500, 17000, 16500, 18000, 19000]

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun",
          "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]

# Max, Min, Average
maximum = sales[0]
minimum = sales[0]
total = 0

for value in sales:
    total += value
    if value > maximum:
        maximum = value
    if value < minimum:
        minimum = value

average = total / len(sales)

print("Maximum Sales:", maximum)
print("Minimum Sales:", minimum)
print("Average Sales:", average)

# Months above average
print("\nMonths with Sales Above Average:")
for i in range(len(sales)):
    if sales[i] > average:
        print(months[i], "-", sales[i])
```

```
# Growth or Decline comparison
print("\nMonthly Growth Analysis:")
for i in range(1, len(sales)):
    if sales[i] > sales[i - 1]:
        print(months[i], ": Growth")
    else:
        print(months[i], ": Decline")
```

Output

```
Maximum Sales: 19000
Minimum Sales: 12000
Average Sales: 15316.666666666666

Months with Sales Above Average:
Jul - 16000
Aug - 15500
Sep - 17000
Oct - 16500
Nov - 18000
Dec - 19000

Monthly Growth Analysis:
Feb : Growth
Mar : Decline
Apr : Growth
May : Growth
Jun : Decline
Jul : Growth
Aug : Decline
Sep : Growth
Oct : Decline
Nov : Growth
Dec : Growth
```

Question 14

```
import random

# Generate multiplication table (1 to 10)
table = []
for i in range(1, 11):
    row = []
    for j in range(1, 11):
        row.append(i * j)
    table.append(row)

# Select 3 random positions to hide
hidden_positions = []
while len(hidden_positions) < 3:
    pos = (random.randint(0, 9), random.randint(0, 9))
    if pos not in hidden_positions:
        hidden_positions.append(pos)

# Show table with blanks
print("Multiplication Puzzle (?? are missing values)\n")
for i in range(10):
    for j in range(10):
        if (i, j) in hidden_positions:
            print("??", end="\t")
        else:
            print(table[i][j], end="\t")
    print()
```

```
# Guessing
score = 0
for pos in hidden_positions:
    i, j = pos
    guess = int(input(f"\nGuess value for {i+1} x {j+1}: "))
    if guess == table[i][j]:
        print("Correct!")
        score += 1
    else:
        print("Wrong! Correct answer is", table[i][j])

# Performance remark
print("\nYour Score:", score, "/ 3")

if score == 3:
    print("Performance: Excellent")
elif score == 2:
    print("Performance: Good")
else:
    print("Performance: Try Again")
```

Output

```
Multiplication Puzzle (?? are missing values)

1       2       3       4       5       6       7       8       9       10
2       4       6       8       10      12      14      16      18      20
??      6       9       12      15      18      21      24      27      30
4       8       12      16      20      24      28      32      36      40
5       10      15      20      25      30      35      40      45      50
6       12      18      24      30      36      42      48      54      60
7       14      21      28      35      42      49      56      63      70
8       16      24      32      40      48      56      64      72      ??
9       18      27      36      45      54      63      72      81      90
??      20      30      40      50      60      70      80      90      100

Guess value for 8 x 10:  80
Correct!

Guess value for 3 x 1:  3
Correct!

Guess value for 10 x 1:  10
Correct!

Your Score: 3 / 3
Performance: Excellent
```

Question 15

```
attempts = 0
max_attempts = 3
special_chars = "!@#$%^&*()_+-=[]{}|;:',.<>?/"

while attempts < max_attempts:
    password = input("Enter password: ")
    attempts += 1

    has_upper = False
    has_lower = False
    has_digit = False
    has_special = False

    for ch in password:
        if ch.isupper():
            has_upper = True
        elif ch.islower():
            has_lower = True
        elif ch.isdigit():
            has_digit = True
        elif ch in special_chars:
            has_special = True

    # Check strength
    if len(password) >= 8 and has_upper and has_lower and has_digit and has_special:
        print("Password is Strong. Access Granted.")
        break
    else:
        print("Weak Password.")
        if attempts < max_attempts:
            print("Please try again.\n")
        else:
            print("\nAccount Locked.")
```

Output

```
Enter password:  tf(reem)
Weak Password.
Please try again.

Enter password:  tfxhk0312:))
Weak Password.
Please try again.

Enter password:  Tfxhk0312:))
Password is Strong. Access Granted.
```

Question 16

```python
def calculate_bill(units, late_payment=False):
    bill = 0

    # Slab calculation
    if units <= 100:
        bill = units * 5
    elif units <= 200:
        bill = (100 * 5) + ((units - 100) * 7)
    else:
        bill = (100 * 5) + (100 * 7) + ((units - 200) * 10)

    penalty = 0
    if late_payment:
        penalty = bill * 0.15

    total_bill = bill + penalty

    # Bill breakdown
    return {
        "Units Used": units,
        "Base Bill": bill,
        "Penalty": penalty,
        "Total Bill": total_bill
    }

# Example usage
units_used = int(input("Enter units consumed: "))
late = input("Late payment? (yes/no): ").lower()

result = calculate_bill(units_used, late == "yes")

print("\n--- Electricity Bill ---")
for key, value in result.items():
    print(key, ":", value)
```

Output

```
Enter units consumed:  35
Late payment? (yes/no):  no

--- Electricity Bill ---
Units Used : 35
Base Bill : 175
Penalty : 0
Total Bill : 175
```

Question 17

```python
def predict_score(runs, balls, wickets, overs_left):
    if balls == 0 or overs_left == 0:
        return "Invalid input"

    strike_rate = (runs / balls) * 100
    run_rate = runs / (20 - overs_left) if overs_left < 20 else runs / (20 - overs_left + 0.0001)

    if run_rate < 5 and wickets > 6:
        return "Losing"
    elif run_rate > 7 and wickets < 5:
        return "Winning"
    else:
        return "Balanced"

# Example
runs = int(input("Runs scored: "))
balls = int(input("Balls faced: "))
wickets = int(input("Wickets lost: "))
overs_left = float(input("Overs left: "))

status = predict_score(runs, balls, wickets, overs_left)
print("Match Status:", status)
```

Output

```
Runs scored:  14
Balls faced:  6
Wickets lost:  3
Overs left:  3
Match Status: Balanced
```

Question 18

```python
def book_room(room_type, days, is_member=False):
    rates = {"standard": 2000, "deluxe": 4000, "suite": 6000}

    if room_type.lower() not in rates:
        return "Invalid room type"

    price = rates[room_type.lower()] * days

    # Discounts
    if days > 7:
        price *= 0.8  # 20% discount
    if is_member:
        price *= 0.9  # extra 10% discount

    tax = price * 0.12
    total = price + tax

    return {
        "Room Type": room_type.capitalize(),
        "Days": days,
        "Member Discount Applied": is_member,
        "Tax": tax,
        "Total Bill": total
    }

# Example
room = input("Room type (Standard/Deluxe/Suite): ")
days = int(input("Number of days: "))
member = input("Member? (yes/no): ").lower() == "yes"

bill = book_room(room, days, member)
print("\n--- Bill Slip ---")
for k, v in bill.items():
    print(k, ":", v)
```

Output

```
Room type (Standard/Deluxe/Suite):  suite
Number of days:  4
Member? (yes/no):  yes

--- Bill Slip ---
Room Type : Suite
Days : 4
Member Discount Applied : True
Tax : 2592.0
Total Bill : 24192.0
```

Question 19

```python
def calculate_fare(distance, class_type, international=False, days_in_advance=0):
    rates = {"economy": 10, "business": 25, "first": 40}

    if class_type.lower() not in rates:
        return "Invalid class type"

    base_fare = rates[class_type.lower()] * distance
    base_fare += 500  # fuel surcharge

    # International tax
    if international:
        base_fare *= 1.18

    # Advance booking discount
    if days_in_advance > 30:
        base_fare *= 0.9

    return base_fare

# Example
distance = float(input("Distance (units): "))
class_type = input("Class type (Economy/Business/First): ")
international = input("International? (yes/no): ").lower() == "yes"
days_advance = int(input("Days in advance: "))

fare = calculate_fare(distance, class_type, international, days_advance)
print("Total Flight Fare:", fare)
```

Output

```
Distance (units):  32
Class type (Economy/Business/First):  first
International? (yes/no):  yes
Days in advance:  3
Total Flight Fare: 2100.4
```

Question 20

```python
def vending_machine(item_name, amount_paid, wallet_balance):
    items = {
        "chips": {"price": 50, "stock": 5},
        "chocolate": {"price": 100, "stock": 3},
        "soda": {"price": 70, "stock": 2}
    }

    if item_name.lower() not in items:
        return "Item not available."

    item = items[item_name.lower()]

    if item["stock"] == 0:
        alternatives = [i for i in items if items[i]["stock"] > 0]
        return f"Item out of stock. Try: {', '.join(alternatives)}"

    if amount_paid < item["price"]:
        return "Insufficient payment."

    # Dispense item
    change = amount_paid - item["price"]
    wallet_balance -= item["price"]
    item["stock"] -= 1

    return {
        "Item Dispensed": item_name.capitalize(),
        "Change Returned": change,
        "Updated Wallet Balance": wallet_balance
```

```python
# Example
wallet = float(input("Wallet balance: "))
item = input("Item to buy: ")
paid = float(input("Amount paid: "))

result = vending_machine(item, paid, wallet)
print("\n--- Vending Machine ---")
print(result)
```

Output

```
Wallet balance:  2311
Item to buy:  soda
Amount paid:  234

--- Vending Machine ---
{'Item Dispensed': 'Soda', 'Change Returned': 164.0, 'Updated Wallet Balance': 2241.0}
```

LAB - 【05】

What is OOP?

OOP is a programming style where we **model real-world things** using:

- **Classes** → blueprints
- **Objects** → real things created from classes

Example:

- Class → *Student*
- Object → *Ali, Sara*

Why OOP is Used

- Organizes code properly
- Reuses code
- Makes programs easy to understand and maintain
- Very useful for **large projects, AI systems, and software design**

Class

A class is a **blueprint** that defines properties and behavior.

```
class Student:
    pass
```

Object

An object is a **real instance** of a class.

```
s1 = Student()
```

Attributes (Variables in Class)

```
class Student:
    name = "Ali"
    marks = 85

s1 = Student()
print(s1.name)
print(s1.marks)
```

Constructor (__init__)

The constructor runs **automatically** when an object is created.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

s1 = Student("Ali", 90)
print(s1.name)
print(s1.marks)
```

Methods (Functions in Class)

```python
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def display(self):
        print("Name:", self.name)
        print("Marks:", self.marks)

s1 = Student("Sara", 88)
s1.display()
```

Four Pillars of OOP

1. Encapsulation

Wrapping data and methods together (**Benefit:** Data protection.)

```python
class Account:
    def __init__(self, balance):
        self.balance = balance

    def show_balance(self):
        print("Balance:", self.balance)
```

2. Inheritance

One class inherits another class.

```python
class Person:
    def show(self):
        print("I am a person")

class Student(Person):
    pass

s = Student()
s.show()
```

3. Polymorphism

Same function name, **different behavior**.

```python
class Bird:
    def sound(self):
        print("Bird sound")

class Dog:
    def sound(self):
        print("Bark")

b = Bird()
d = Dog()
b.sound()
d.sound()
```

4. Abstraction

Hiding unnecessary details.

```python
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Square(Shape):
    def area(self):
        print("Area of square")

s = Square()
s.area()
```

Access Modifiers in Python

Type	Syntax
Public	name
Protected	_name
Private	__name

OOP vs Normal Programming

Normal	OOP
Functions	Objects
Less secure	More secure
Not reusable	Reusable

Lab Exercise

Question 1

```
class BankAccount:
    def __init__(self, account_number, account_holder, balance=0.0):
        self.account_number = account_number
        self.account_holder = account_holder
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited: {amount}")
        else:
            print("Deposit amount must be positive!")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient balance!")
        elif amount <= 0:
            print("Withdrawal amount must be positive!")
        else:
            self.balance -= amount
            print(f"Withdrawn: {amount}")

    def display_balance(self):
        print(f"Account Holder: {self.account_holder}")
        print(f"Account Number: {self.account_number}")
        print(f"Current Balance: {self.balance}")


# Example usage:
account1 = BankAccount("123456", "Tehreem Fatima", 1000)
```

```
# Example usage:
account1 = BankAccount("123456", "Tehreem Fatima", 1000)

account1.display_balance()
account1.deposit(500)
account1.withdraw(300)
account1.withdraw(1500)  # trying to withdraw more than balance
account1.display_balance()
```

```
Account Holder: Tehreem Fatima
Account Number: 123456
Current Balance: 1000
Deposited: 500
Withdrawn: 300
Insufficient balance!
Account Holder: Tehreem Fatima
Account Number: 123456
Current Balance: 1200
```

Question 2

```
class Book:
    def __init__(self, title, author, isbn):
        self.title = title
        self.author = author
        self.isbn = isbn
        self.available = True  # True = available, False = checked out

    def display_info(self):
        status = "Available" if self.available else "Checked Out"
        print(f"Title: {self.title}, Author: {self.author}, ISBN: {self.isbn}, Status: {status}")


class Library:
    def __init__(self):
        self.books = []  # list to store all books

    def add_book(self, book):
        self.books.append(book)
        print(f"Book '{book.title}' added to the library.")

    def search_by_title(self, title):
        print(f"\nSearching for books with title '{title}':")
        found = False
        for book in self.books:
            if book.title.lower() == title.lower():
                book.display_info()
                found = True
        if not found:
            print("No book found with that title.")
```

```
    def search_by_author(self, author):
        print(f"\nSearching for books by author '{author}':")
        found = False
        for book in self.books:
            if book.author.lower() == author.lower():
                book.display_info()
                found = True
        if not found:
            print("No books found by that author.")

    def checkout_book(self, title):
        for book in self.books:
            if book.title.lower() == title.lower():
                if book.available:
                    book.available = False
                    print(f"You have checked out '{book.title}'.")
                else:
                    print(f"'{book.title}' is already checked out.")
                return
        print("Book not found.")

    def return_book(self, title):
        for book in self.books:
            if book.title.lower() == title.lower():
                if not book.available:
                    book.available = True
                    print(f"You have returned '{book.title}'.")
                else:
                    print(f"'{book.title}' was not checked out.")
                return
        print("Book not found.")
```

```
    def display_all_books(self):
        print("\nAll Books in Library:")
        if not self.books:
            print("No books in the library.")
        else:
            for book in self.books:
                book.display_info()


# Example usage
library = Library()

# Adding books
book1 = Book("Harry Potter", "J.K. Rowling", "12345")
book2 = Book("The Alchemist", "Paulo Coelho", "67890")
book3 = Book("1984", "George Orwell", "54321")

library.add_book(book1)
library.add_book(book2)
library.add_book(book3)

# Searching and checking out
library.search_by_author("J.K. Rowling")
library.checkout_book("Harry Potter")
library.display_all_books()

# Returning the book
library.return_book("Harry Potter")
library.display_all_books()
```

```
Book 'Harry Potter' added to the library.
Book 'The Alchemist' added to the library.
Book '1984' added to the library.

Searching for books by author 'J.K. Rowling':
Title: Harry Potter, Author: J.K. Rowling, ISBN: 12345, Status: Available
You have checked out 'Harry Potter'.

All Books in Library:
Title: Harry Potter, Author: J.K. Rowling, ISBN: 12345, Status: Checked Out
Title: The Alchemist, Author: Paulo Coelho, ISBN: 67890, Status: Available
Title: 1984, Author: George Orwell, ISBN: 54321, Status: Available
You have returned 'Harry Potter'.

All Books in Library:
Title: Harry Potter, Author: J.K. Rowling, ISBN: 12345, Status: Available
Title: The Alchemist, Author: Paulo Coelho, ISBN: 67890, Status: Available
Title: 1984, Author: George Orwell, ISBN: 54321, Status: Available
```

Question 3

8]:
```
class Student:
    def __init__(self, student_id, name):
        self.student_id = student_id
        self.name = name
        self.grades = []  # list to store grades

    def add_grade(self, grade):
        if 0 <= grade <= 100:  # grade must be between 0 and 100
            self.grades.append(grade)
            print(f"Grade {grade} added for {self.name}.")
        else:
            print("Invalid grade! Please enter a value between 0 and 100.")

    def calculate_average(self):
        if len(self.grades) == 0:
            return 0
        return sum(self.grades) / len(self.grades)

    def get_letter_grade(self):
        average = self.calculate_average()
        if average >= 90:
            return "A"
        elif average >= 80:
            return "B"
        elif average >= 70:
            return "C"
        elif average >= 60:
            return "D"
        else:
            return "F"
```

```
    def display_info(self):
        print("\n--- Student Performance Summary ---")
        print(f"Student ID: {self.student_id}")
        print(f"Name: {self.name}")
        print(f"Grades: {self.grades}")
        average = self.calculate_average()
        print(f"Average Grade: {average:.2f}")
        print(f"Letter Grade: {self.get_letter_grade()}")


# Example usage
student1 = Student("S101", "Tehreem Fatima")

student1.add_grade(95)
student1.add_grade(85)
student1.add_grade(78)

student1.display_info()
```

Output

```
Grade 95 added for Tehreem Fatima.
Grade 85 added for Tehreem Fatima.
Grade 78 added for Tehreem Fatima.

--- Student Performance Summary ---
Student ID: S101
Name: Tehreem Fatima
Grades: [95, 85, 78]
Average Grade: 86.00
Letter Grade: B
```

Question 4

```
]:
class MenuItem:
    def __init__(self, name, price, category):
        self.name = name
        self.price = price
        self.category = category

    def display_item(self):
        print(f"{self.name} ({self.category}) - Rs.{self.price:.2f}")


class Order:
    def __init__(self):
        self.items = []  # list to store added menu items
        self.tax_rate = 0.08  # 8% tax

    def add_item(self, item):
        self.items.append(item)
        print(f"Added: {item.name} - Rs.{item.price:.2f}")

    def calculate_subtotal(self):
        return sum(item.price for item in self.items)

    def calculate_tax(self):
        return self.calculate_subtotal() * self.tax_rate

    def apply_discount(self, discount_percent):
        subtotal = self.calculate_subtotal()
        discount_amount = subtotal * (discount_percent / 100)
        return subtotal - discount_amount
```

```
    def calculate_total(self, discount_percent=0):
        discounted_total = self.apply_discount(discount_percent)
        total_with_tax = discounted_total + (discounted_total * self.tax_rate)
        return total_with_tax

    def generate_receipt(self, discount_percent=0):
        print("\n----- RECEIPT -----")
        for item in self.items:
            item.display_item()

        subtotal = self.calculate_subtotal()
        discount_amount = subtotal * (discount_percent / 100)
        tax = self.calculate_tax()
        total = self.calculate_total(discount_percent)

        print(f"\nSubtotal: Rs.{subtotal:.2f}")
        if discount_percent > 0:
            print(f"Discount ({discount_percent}%): -Rs.{discount_amount:.2f}")
        print(f"Tax (8%): Rs.{tax:.2f}")
        print(f"Total: Rs.{total:.2f}")
        print("--------------------")


# Example usage
item1 = MenuItem("Burger", 350, "Fast Food")
item2 = MenuItem("Pizza", 800, "Fast Food")
item3 = MenuItem("Cold Drink", 120, "Beverage")

order = Order()
order.add_item(item1)
order.add_item(item2)
order.add_item(item3)

order.generate_receipt(discount_percent=10)
```

Output

```
Added: Burger - Rs.350.00
Added: Pizza - Rs.800.00
Added: Cold Drink - Rs.120.00

----- RECEIPT -----
Burger (Fast Food) - Rs.350.00
Pizza (Fast Food) - Rs.800.00
Cold Drink (Beverage) - Rs.120.00

Subtotal: Rs.1270.00
Discount (10%): -Rs.127.00
Tax (8%): Rs.101.60
Total: Rs.1234.44
```

Question 5

[4]:
```python
class Vehicle:
    def __init__(self, license_plate, model, rental_price_per_day):
        self.license_plate = license_plate
        self.model = model
        self.rental_price_per_day = rental_price_per_day
        self.available = True  # available for rent by default

    def rent_vehicle(self, days):
        if self.available:
            self.available = False
            total_cost = self.rental_price_per_day * days
            print(f"Vehicle {self.model} (License: {self.license_plate}) rented for {days} days.")
            print(f"Total Cost: Rs.{total_cost:.2f}")
        else:
            print(f"Sorry, {self.model} is currently not available.")

    def return_vehicle(self):
        if not self.available:
            self.available = True
            print(f"Vehicle {self.model} (License: {self.license_plate}) has been returned.")
        else:
            print(f"Vehicle {self.model} was not rented.")

    def display_info(self):
        status = "Available" if self.available else "Rented"
        print(f"Model: {self.model}, License: {self.license_plate}, Price/Day: Rs.{self.rental_price_per_day}, Status: {status}")


# Derived classes
class Car(Vehicle):
    def __init__(self, license_plate, model, rental_price_per_day, seats):
        super().__init__(license_plate, model, rental_price_per_day)
        self.seats = seats
```

```python
class Car(Vehicle):
    def __init__(self, license_plate, model, rental_price_per_day, seats):
        super().__init__(license_plate, model, rental_price_per_day)
        self.seats = seats

    def display_info(self):
        super().display_info()
        print(f"Seats: {self.seats}")


class Motorcycle(Vehicle):
    def __init__(self, license_plate, model, rental_price_per_day, engine_cc):
        super().__init__(license_plate, model, rental_price_per_day)
        self.engine_cc = engine_cc

    def display_info(self):
        super().display_info()
        print(f"Engine: {self.engine_cc}cc")


class Truck(Vehicle):
    def __init__(self, license_plate, model, rental_price_per_day, load_capacity):
        super().__init__(license_plate, model, rental_price_per_day)
        self.load_capacity = load_capacity  # in tons

    def display_info(self):
        super().display_info()
        print(f"Load Capacity: {self.load_capacity} tons")


# Example usage
car1 = Car("ABC-123", "Toyota Corolla", 5000, 5)
bike1 = Motorcycle("XYZ-789", "Honda 125", 1500, 125)
truck1 = Truck("TRK-555", "Hino 300", 10000, 5)
```

```python
print("\n--- Vehicle Information ---")
car1.display_info()
bike1.display_info()
truck1.display_info()

print("\n--- Renting Vehicles ---")
car1.rent_vehicle(3)
bike1.rent_vehicle(2)
truck1.rent_vehicle(5)

print("\n--- Returning Vehicles ---")
car1.return_vehicle()
truck1.return_vehicle()

print("\n--- Updated Status ---")
car1.display_info()
bike1.display_info()
truck1.display_info()
```

Output

```
--- Vehicle Information ---
Model: Toyota Corolla, License: ABC-123, Price/Day: Rs.5000, Status: Available
Seats: 5
Model: Honda 125, License: XYZ-789, Price/Day: Rs.1500, Status: Available
Engine: 125cc
Model: Hino 300, License: TRK-555, Price/Day: Rs.10000, Status: Available
Load Capacity: 5 tons

--- Renting Vehicles ---
Vehicle Toyota Corolla (License: ABC-123) rented for 3 days.
Total Cost: Rs.15000.00
Vehicle Honda 125 (License: XYZ-789) rented for 2 days.
Total Cost: Rs.3000.00
Vehicle Hino 300 (License: TRK-555) rented for 5 days.
Total Cost: Rs.50000.00

--- Returning Vehicles ---
Vehicle Toyota Corolla (License: ABC-123) has been returned.
Vehicle Hino 300 (License: TRK-555) has been returned.

--- Updated Status ---
Model: Toyota Corolla, License: ABC-123, Price/Day: Rs.5000, Status: Available
Seats: 5
Model: Honda 125, License: XYZ-789, Price/Day: Rs.1500, Status: Rented
Engine: 125cc
Model: Hino 300, License: TRK-555, Price/Day: Rs.10000, Status: Available
Load Capacity: 5 tons
```

Question 6

```
7]: from datetime import date

class Employee:
    def __init__(self, emp_id, name, position, salary, start_year):
        self.emp_id = emp_id
        self.name = name
        self.position = position
        self.salary = salary  # annual salary
        self.start_year = start_year  # year employee joined

    def calculate_monthly_salary(self):
        monthly_salary = self.salary / 12
        return monthly_salary

    def calculate_annual_salary(self):
        return self.salary

    def apply_raise(self, percent):
        if percent > 0:
            raise_amount = self.salary * (percent / 100)
            self.salary += raise_amount
            print(f"Salary increased by {percent}% for {self.name}. New salary: Rs.{self.salary:.2f}")
        else:
            print("Raise percentage must be positive.")

    def calculate_employment_duration(self):
        current_year = date.today().year
        duration = current_year - self.start_year
        return duration

    def display_info(self):
        print("\n--- Employee Information ---")
        print(f"Employee ID: {self.emp_id}")
        print(f"Name: {self.name}")
```

```
    def display_info(self):
        print("\n--- Employee Information ---")
        print(f"Employee ID: {self.emp_id}")
        print(f"Name: {self.name}")
        print(f"Position: {self.position}")
        print(f"Annual Salary: Rs.{self.salary:.2f}")
        print(f"Monthly Salary: Rs.{self.calculate_monthly_salary():.2f}")
        print(f"Employment Duration: {self.calculate_employment_duration()} years")


# Example usage
emp1 = Employee("E101", "Tehreem Fatima", "Software Engineer", 1200000, 2020)
emp2 = Employee("E102", "Ali Khan", "Data Analyst", 900000, 2022)

emp1.display_info()
emp2.display_info()

print("\n--- Applying Raises ---")
emp1.apply_raise(10)
emp2.apply_raise(5)

emp1.display_info()
emp2.display_info()
```

Output

```
--- Employee Information ---
Employee ID: E101
Name: Tehreem Fatima
Position: Software Engineer
Annual Salary: Rs.1200000.00
Monthly Salary: Rs.100000.00
Employment Duration: 5 years

--- Employee Information ---
Employee ID: E102
Name: Ali Khan
Position: Data Analyst
Annual Salary: Rs.900000.00
Monthly Salary: Rs.75000.00
Employment Duration: 3 years

--- Applying Raises ---
Salary increased by 10% for Tehreem Fatima. New salary: Rs.1320000.00
Salary increased by 5% for Ali Khan. New salary: Rs.945000.00

--- Employee Information ---
Employee ID: E101
Name: Tehreem Fatima
Position: Software Engineer
Annual Salary: Rs.1320000.00
Monthly Salary: Rs.110000.00
Employment Duration: 5 years

--- Employee Information ---
Employee ID: E102
Name: Ali Khan
Position: Data Analyst
Annual Salary: Rs.945000.00
Monthly Salary: Rs.78750.00
Employment Duration: 3 years
```

Question 7

0]:

```python
class Product:
    def __init__(self, product_id, name, price, stock_quantity):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.stock_quantity = stock_quantity

    def update_stock(self, quantity):
        self.stock_quantity -= quantity

    def restock(self, quantity):
        self.stock_quantity += quantity

    def display_info(self):
        print(f"ID: {self.product_id}, Name: {self.name}, Price: Rs.{self.price:.2f}, Stock: {self.stock_quantity}")


class ShoppingCart:
    def __init__(self):
        self.cart = {}  # {product: quantity}

    def add_product(self, product, quantity):
        if product.stock_quantity >= quantity:
            if product in self.cart:
                self.cart[product] += quantity
            else:
                self.cart[product] = quantity
            product.update_stock(quantity)
            print(f"Added {quantity} x {product.name} to cart.")
        else:
            print(f"Not enough stock for {product.name}. Available: {product.stock_quantity}")
```

```
    def remove_product(self, product, quantity):
        if product in self.cart:
            if quantity >= self.cart[product]:
                removed_quantity = self.cart.pop(product)
                product.restock(removed_quantity)
                print(f"Removed all {removed_quantity} x {product.name} from cart.")
            else:
                self.cart[product] -= quantity
                product.restock(quantity)
                print(f"Removed {quantity} x {product.name} from cart.")
        else:
            print(f"{product.name} is not in the cart.")

    def calculate_total(self):
        total = sum(product.price * quantity for product, quantity in self.cart.items())
        return total

    def checkout(self):
        if not self.cart:
            print("Your cart is empty!")
            return
        total = self.calculate_total()
        print("\n--- Checkout Summary ---")
        for product, quantity in self.cart.items():
            print(f"{product.name} x {quantity} = Rs.{product.price * quantity:.2f}")
        print(f"Total Amount: Rs.{total:.2f}")
        print("Thank you for your purchase!")
        self.cart.clear()

    def display_cart(self):
        if not self.cart:
            print("Cart is empty.")
        else:
            print("\n--- Shopping Cart ---")
            for product, quantity in self.cart.items():
                print(f"{product.name} x {quantity} = Rs.{product.price * quantity:.2f}")
```

```
# Example usage
p1 = Product("P001", "Laptop", 150000, 5)
p2 = Product("P002", "Headphones", 4000, 10)
p3 = Product("P003", "Mouse", 1500, 15)

cart = ShoppingCart()

# Adding products
cart.add_product(p1, 1)
cart.add_product(p2, 2)
cart.add_product(p3, 3)

cart.display_cart()

# Removing product
cart.remove_product(p2, 1)
cart.display_cart()

# Checkout
cart.checkout()

# Display updated stock
print("\n--- Updated Product Stock ---")
p1.display_info()
p2.display_info()
p3.display_info()
```

Output

```
Added 1 x Laptop to cart.
Added 2 x Headphones to cart.
Added 3 x Mouse to cart.

--- Shopping Cart ---
Laptop x 1 = Rs.150000.00
Headphones x 2 = Rs.8000.00
Mouse x 3 = Rs.4500.00
Total: Rs.162500.00
Removed 1 x Headphones from cart.

--- Shopping Cart ---
Laptop x 1 = Rs.150000.00
Headphones x 1 = Rs.4000.00
Mouse x 3 = Rs.4500.00
Total: Rs.158500.00

--- Checkout Summary ---
Laptop x 1 = Rs.150000.00
Headphones x 1 = Rs.4000.00
Mouse x 3 = Rs.4500.00
Total Amount: Rs.158500.00
Thank you for your purchase!

--- Updated Product Stock ---
ID: P001, Name: Laptop, Price: Rs.150000.00, Stock: 4
ID: P002, Name: Headphones, Price: Rs.4000.00, Stock: 9
ID: P003, Name: Mouse, Price: Rs.1500.00, Stock: 12
```

Question 8

[23]:
```python
class Patient:
    def __init__(self, patient_id, name, age, medical_history=None):
        self.patient_id = patient_id
        self.name = name
        self.age = age
        self.medical_history = medical_history if medical_history else []

    def add_medical_record(self, record):
        self.medical_history.append(record)
        print(f"Added medical record for {self.name}: {record}")

    def display_info(self):
        print(f"\n--- Patient Information ---")
        print(f"ID: {self.patient_id}")
        print(f"Name: {self.name}")
        print(f"Age: {self.age}")
        print(f"Medical History: {', '.join(self.medical_history) if self.medical_history else 'No records available.'}")


class Appointment:
    def __init__(self):
        self.appointments = []  # list of dictionaries

    def schedule_appointment(self, patient, doctor_name, date, time):
        appointment = {
            "patient_id": patient.patient_id,
            "patient_name": patient.name,
            "doctor": doctor_name,
            "date": date,
            "time": time
        }
        self.appointments.append(appointment)
        print(f"Appointment scheduled for {patient.name} with Dr. {doctor_name} on {date} at {time}.")
```

```python
    def cancel_appointment(self, patient_id, date, time):
        for appt in self.appointments:
            if appt["patient_id"] == patient_id and appt["date"] == date and appt["time"] == time:
                self.appointments.remove(appt)
                print(f"Appointment for {appt['patient_name']} on {date} at {time} has been cancelled.")
                return
        print("No matching appointment found to cancel.")

    def view_appointments(self):
        if not self.appointments:
            print("No appointments scheduled.")
        else:
            print("\n--- All Appointments ---")
            for appt in self.appointments:
                print(f"Patient: {appt['patient_name']} | Doctor: Dr. {appt['doctor']} | Date: {appt['date']} | Time: {appt['time']}")


# Example usage
p1 = Patient("P001", "Tehreem Fatima", 22, ["Allergy - 2022"])
p2 = Patient("P002", "Ali Khan", 30)

# Display patient info
p1.display_info()
p2.display_info()

# Add medical record
p2.add_medical_record("Flu - 2023")

# Appointment system
hospital = Appointment()

hospital.schedule_appointment(p1, "Dr. Ahmed", "2025-10-20", "11:00 AM")
hospital.schedule_appointment(p2, "Dr. Sara", "2025-10-21", "2:00 PM")

hospital.view_appointments()
```

```python
# Cancel an appointment
hospital.cancel_appointment("P002", "2025-10-21", "2:00 PM")

hospital.view_appointments()
```

Output

```
--- Patient Information ---
ID: P001
Name: Tehreem Fatima
Age: 22
Medical History: Allergy - 2022

--- Patient Information ---
ID: P002
Name: Ali Khan
Age: 30
Medical History: No records available.
Added medical record for Ali Khan: Flu - 2023
Appointment scheduled for Tehreem Fatima with Dr. Dr. Ahmed on 2025-10-20 at 11:00 AM.
Appointment scheduled for Ali Khan with Dr. Dr. Sara on 2025-10-21 at 2:00 PM.

--- All Appointments ---
Patient: Tehreem Fatima | Doctor: Dr. Dr. Ahmed | Date: 2025-10-20 | Time: 11:00 AM
Patient: Ali Khan | Doctor: Dr. Dr. Sara | Date: 2025-10-21 | Time: 2:00 PM
Appointment for Ali Khan on 2025-10-21 at 2:00 PM has been cancelled.

--- All Appointments ---
Patient: Tehreem Fatima | Doctor: Dr. Dr. Ahmed | Date: 2025-10-20 | Time: 11:00 AM
```

Question 9

```
class Post:
    def __init__(self, content, privacy="Public"):
        self.content = content
        self.privacy = privacy  # "Public" or "Friends Only"

    def display_post(self, viewer_is_friend=False):
        if self.privacy == "Public" or viewer_is_friend:
            print(f"Post: {self.content} ({self.privacy})")
        else:
            print("This post is private.")


class User:
    def __init__(self, username, email):
        self.username = username
        self.email = email
        self.friends = []
        self.posts = []

    def add_friend(self, friend_user):
        if friend_user not in self.friends:
            self.friends.append(friend_user)
            friend_user.friends.append(self)
            print(f"{self.username} and {friend_user.username} are now friends.")
        else:
            print(f"{friend_user.username} is already your friend.")
```

```
    def create_post(self, content, privacy="Public"):
        new_post = Post(content, privacy)
        self.posts.append(new_post)
        print(f"{self.username} created a new {privacy} post.")

    def display_timeline(self, viewer_user=None):
        print(f"\n--- {self.username}'s Timeline ---")
        for post in self.posts:
            is_friend = viewer_user in self.friends if viewer_user else False
            post.display_post(viewer_is_friend=is_friend)


# Example usage
user1 = User("Tehreem", "tehreem@example.com")
user2 = User("Ali", "ali@example.com")
user3 = User("Sara", "sara@example.com")

# Adding friends
user1.add_friend(user2)

# Creating posts
user1.create_post("Just had coffee ", "Public")
user1.create_post("Feeling lazy today ", "Friends Only")
user2.create_post("Working on a new project ", "Public")

# Viewing timelines
user1.display_timeline()            # Tehreem viewing own posts
user1.display_timeline(user2)       # Ali viewing Tehreem's posts
user1.display_timeline(user3)       # Sara (not friend) viewing Tehreem's posts
```

Output

```
Tehreem and Fatima are now friends.
Tehreem created a new Public post.
Tehreem created a new Friends Only post.
Fatima created a new Public post.

--- Tehreem's Timeline ---
Post: Just had coffee  (Public)
This post is private.

--- Tehreem's Timeline ---
Post: Just had coffee  (Public)
Post: Feeling lazy today  (Friends Only)

--- Tehreem's Timeline ---
Post: Just had coffee  (Public)
This post is private.
```

Question 10

[33]:

```python
class Room:
    def __init__(self, room_number, room_type, price, available=True):
        self.room_number = room_number
        self.room_type = room_type
        self.price = price
        self.available = available

    def display_info(self):
        print(f"Room {self.room_number} | Type: {self.room_type} | Price per Night: {self.price} | Available: {self.available}")


class Booking:
    def __init__(self, guest_name, room, check_in, check_out):
        self.guest_name = guest_name
        self.room = room
        self.check_in = check_in
        self.check_out = check_out
        self.days = (check_out - check_in).days

    def calculate_total_cost(self):
        return self.room.price * self.days

    def confirm_booking(self):
        if self.room.available:
            self.room.available = False
            print(f"Booking confirmed for {self.guest_name} in Room {self.room.room_number}.")
        else:
            print("Sorry, the room is not available.")

    def cancel_booking(self):
        self.room.available = True
        print(f"Booking cancelled for {self.guest_name}.")
```

```
    def cancel_booking(self):
        self.room.available = True
        print(f"Booking cancelled for {self.guest_name}.")

    def display_booking_details(self):
        print(f"Guest: {self.guest_name}")
        print(f"Room Number: {self.room.room_number}")
        print(f"Stay Duration: {self.days} days")
        print(f"Total Cost: {self.calculate_total_cost()}")


# Example usage
if __name__ == "__main__":
    from datetime import date

    # Create rooms
    room1 = Room(101, "Single", 5000)
    room2 = Room(202, "Suite", 10000)

    # Display available rooms
    room1.display_info()
    room2.display_info()

    # Make a booking
    booking1 = Booking("Ayesha", room1, date(2025, 10, 14), date(2025, 10, 17))
    booking1.confirm_booking()
    booking1.display_booking_details()

    # Cancel booking
    booking1.cancel_booking()
    room1.display_info()
```

Output

```
Room 101 | Type: Single | Price per Night: 5000 | Available: True
Room 202 | Type: Suite | Price per Night: 10000 | Available: True
Booking confirmed for Ayesha in Room 101.
Guest: Ayesha
Room Number: 101
Stay Duration: 3 days
Total Cost: 15000
Booking cancelled for Ayesha.
Room 101 | Type: Single | Price per Night: 5000 | Available: True
```

LAB - 【07】

Question 11

```
class Course:
    def __init__(self, code, name, credits, capacity, schedule):
        self.code = code
        self.name = name
        self.credits = credits
        self.capacity = capacity
        self.schedule = schedule  # e.g. "Mon 9-11"
        self.students = []

    def register_student(self, student):
        if len(self.students) < self.capacity:
            self.students.append(student)
            print(f"{student.name} has been registered in {self.name}.")
        else:
            print(f"Course {self.name} is full.")

    def display_info(self):
        print(f"{self.code} - {self.name} | Credits: {self.credits} | Capacity: {len(self.students)}/{self.capacity} | Schedule: {self.schedule}")


class Student:
    def __init__(self, student_id, name, max_credits=18):
        self.student_id = student_id
        self.name = name
        self.max_credits = max_credits
        self.courses = []

    def total_credits(self):
        return sum(course.credits for course in self.courses)
```

```
    def has_conflict(self, new_course):
        for course in self.courses:
            if course.schedule == new_course.schedule:
                return True
        return False

    def register_course(self, course):
        if self.has_conflict(course):
            print(f"Cannot register {self.name} in {course.name}: Schedule conflict with another course.")
        elif self.total_credits() + course.credits > self.max_credits:
            print(f"Cannot register {self.name}: Credit limit exceeded.")
        elif len(course.students) >= course.capacity:
            print(f"Cannot register {self.name}: {course.name} is full.")
        else:
            self.courses.append(course)
            course.register_student(self)

    def display_courses(self):
        print(f"Courses registered by {self.name}:")
        for c in self.courses:
            print(f"- {c.name} ({c.code})")


# Example usage
if __name__ == "__main__":
    # Create some courses
    c1 = Course("CS101", "Intro to Programming", 3, 2, "Mon 9-11")
    c2 = Course("CS102", "Data Structures", 4, 2, "Mon 9-11")
    c3 = Course("CS103", "Databases", 3, 2, "Tue 10-12")

    # Create students
    s1 = Student("ST001", "Sara")
    s2 = Student("ST002", "Fatima")
```

```python
# Example usage
if __name__ == "__main__":
    # Create some courses
    c1 = Course("CS101", "Intro to Programming", 3, 2, "Mon 9-11")
    c2 = Course("CS102", "Data Structures", 4, 2, "Mon 9-11")
    c3 = Course("CS103", "Databases", 3, 2, "Tue 10-12")

    # Create students
    s1 = Student("ST001", "Sara")
    s2 = Student("ST002", "Fatima")

    # Register students
    s1.register_course(c1)
    s1.register_course(c2)  # schedule conflict
    s1.register_course(c3)

    s2.register_course(c1)
    s2.register_course(c3)
    s2.register_course(c2)  # full capacity or conflict check

    # Display info
    s1.display_courses()
    s2.display_courses()

    c1.display_info()
    c3.display_info()
```

Output

```
Sara has been registered in Intro to Programming.
Cannot register Sara in Data Structures: Schedule conflict with another course.
Sara has been registered in Databases.
Fatima has been registered in Intro to Programming.
Fatima has been registered in Databases.
Cannot register Fatima in Data Structures: Schedule conflict with another course.
Courses registered by Sara:
- Intro to Programming (CS101)
- Databases (CS103)
Courses registered by Fatima:
- Intro to Programming (CS101)
- Databases (CS103)
CS101 - Intro to Programming | Credits: 3 | Capacity: 2/2 | Schedule: Mon 9-11
CS103 - Databases | Credits: 3 | Capacity: 2/2 | Schedule: Tue 10-12
```

Question 12

```
class Product:
    def __init__(self, product_id, name, quantity, price, supplier):
        self.product_id = product_id
        self.name = name
        self.quantity = quantity
        self.price = price
        self.supplier = supplier

    def update_quantity(self, amount):
        self.quantity += amount
        print(f"Updated quantity for {self.name}. New quantity: {self.quantity}")

    def display_info(self):
        print(f"ID: {self.product_id} | Name: {self.name} | Quantity: {self.quantity} | Price: {self.price} | Supplier: {self.supplier}")


class Inventory:
    def __init__(self):
        self.products = []

    def add_product(self, product):
        self.products.append(product)
        print(f"Product '{product.name}' added to inventory.")

    def update_stock(self, product_id, amount):
        for product in self.products:
            if product.product_id == product_id:
                product.update_quantity(amount)
                return
        print("Product not found in inventory.")
```

```
    def display_all_products(self):
        print("\n--- Inventory List ---")
        for product in self.products:
            product.display_info()

    def low_stock_alert(self, threshold=5):
        print("\n--- Low Stock Alert ---")
        for product in self.products:
            if product.quantity <= threshold:
                print(f"Low stock: {product.name} (Quantity: {product.quantity})")


# Example usage
if __name__ == "__main__":
    # Creating product objects using variations of your name
    p1 = Product("P101", "Tehreem Perfume", 10, 1500, "Tehreem Suppliers")
    p2 = Product("P102", "Tehreem Candles", 3, 800, "Tehreem Creations")
    p3 = Product("P103", "Tehreem Notebook", 2, 250, "Tehreem Stationery")

    # Creating inventory
    inventory = Inventory()
    inventory.add_product(p1)
    inventory.add_product(p2)
    inventory.add_product(p3)

    # Display all products
    inventory.display_all_products()

    # Update stock
    inventory.update_stock("P103", 5)

    # Check low stock
    inventory.low_stock_alert()
```

Output

```
Product 'Tehreem Perfume' added to inventory.
Product 'Tehreem Candles' added to inventory.
Product 'Tehreem Notebook' added to inventory.

--- Inventory List ---
ID: P101 | Name: Tehreem Perfume | Quantity: 10 | Price: 1500 | Supplier: Tehreem Suppliers
ID: P102 | Name: Tehreem Candles | Quantity: 3 | Price: 800 | Supplier: Tehreem Creations
ID: P103 | Name: Tehreem Notebook | Quantity: 2 | Price: 250 | Supplier: Tehreem Stationery
Updated quantity for Tehreem Notebook. New quantity: 7

--- Low Stock Alert ---
Low stock: Tehreem Candles (Quantity: 3)
```

Question 13

```python
class Movie:
    def __init__(self, title, genre, duration, showtimes):
        self.title = title
        self.genre = genre
        self.duration = duration   # in minutes
        self.showtimes = showtimes # list of available times

    def display_info(self):
        print(f"Title: {self.title}")
        print(f"Genre: {self.genre}")
        print(f"Duration: {self.duration} minutes")
        print(f"Showtimes: {', '.join(self.showtimes)}")


class Theater:
    def __init__(self, name, rows, cols):
        self.name = name
        self.rows = rows
        self.cols = cols
        self.seats = [["O" for _ in range(cols)] for _ in range(rows)]  # O = open seat
        self.ticket_prices = {"standard": 500, "premium": 800}

    def display_seating(self):
        print(f"\n--- Seating for {self.name} ---")
        for r in range(self.rows):
            print(" ".join(self.seats[r]))
        print("(O = Available, X = Booked)")

    def reserve_seat(self, row, col, category="standard"):
        if row < 0 or row >= self.rows or col < 0 or col >= self.cols:
            print("Invalid seat number.")
            return
        if self.seats[row][col] == "X":
            print("Seat already booked.")
```

```
        else:
            self.seats[row][col] = "X"
            price = self.ticket_prices.get(category, 500)
            print(f"Seat ({row+1}, {col+1}) booked successfully in {category.title()} category.")
            print(f"Ticket Price: Rs {price}")

    def calculate_total(self, booked_seats):
        total = 0
        for seat in booked_seats:
            category = seat[2]
            total += self.ticket_prices.get(category, 500)
        return total


# Example usage
if __name__ == "__main__":
    # Create a movie
    movie1 = Movie("The Secret of Tehreem", "Mystery", 120, ["1:00 PM", "4:00 PM", "7:00 PM"])
    movie1.display_info()

    # Create a theater
    theater = Theater("Tehreem Cinema Hall", 5, 6)

    # Display available seats
    theater.display_seating()

    # Book a few seats
    theater.reserve_seat(1, 2, "standard")
    theater.reserve_seat(2, 3, "premium")
    theater.reserve_seat(1, 2, "premium")  # already booked

    # Show seating after booking
    theater.display_seating()
```

```
    # Example of total cost for multiple booked seats
    booked = [(1, 2, "standard"), (2, 3, "premium")]
    print("\nTotal Amount:", theater.calculate_total(booked))
```

Output

```
Title: The Secret of Tehreem
Genre: Mystery
Duration: 120 minutes
Showtimes: 1:00 PM, 4:00 PM, 7:00 PM

--- Seating for Tehreem Cinema Hall ---
O O O O O O
O O O O O O
O O O O O O
O O O O O O
O O O O O O
(O = Available, X = Booked)
Seat (2, 3) booked successfully in Standard category.
Ticket Price: Rs 500
Seat (3, 4) booked successfully in Premium category.
Ticket Price: Rs 800
Seat already booked.

--- Seating for Tehreem Cinema Hall ---
O O O O O O
O O X O O O
O O O X O O
O O O O O O
O O O O O O
(O = Available, X = Booked)

Total Amount: 1300
```

Question 14

5]:

```python
class Member:
    def __init__(self, member_id, name, membership_type):
        self.member_id = member_id
        self.name = name
        self.membership_type = membership_type  # "basic" or "premium"
        self.attendance = []  # list to record attendance dates
        self.sessions = []    # list to store scheduled sessions

    def mark_attendance(self, date):
        self.attendance.append(date)
        print(f"Attendance marked for {self.name} on {date}.")

    def display_info(self):
        print(f"Member ID: {self.member_id}")
        print(f"Name: {self.name}")
        print(f"Membership Type: {self.membership_type}")
        print(f"Total Attendance: {len(self.attendance)} days")

    def display_sessions(self):
        if not self.sessions:
            print(f"No sessions scheduled for {self.name}.")
        else:
            print(f"Training Sessions for {self.name}:")
            for s in self.sessions:
                print(f"- {s['trainer']} on {s['date']} at {s['time']}")


class Trainer:
    def __init__(self, trainer_id, name, specialty):
        self.trainer_id = trainer_id
        self.name = name
        self.specialty = specialty
        self.sessions = []  # list to track scheduled sessions
```

```
    def schedule_session(self, member, date, time):
        session = {"member": member.name, "trainer": self.name, "date": date, "time": time}
        self.sessions.append(session)
        member.sessions.append(session)
        print(f"Session scheduled: {member.name} with Trainer {self.name} on {date} at {time}")

    def display_info(self):
        print(f"Trainer ID: {self.trainer_id}")
        print(f"Name: {self.name}")
        print(f"Specialty: {self.specialty}")

    def display_sessions(self):
        if not self.sessions:
            print(f"No sessions scheduled for Trainer {self.name}.")
        else:
            print(f"Training Sessions for {self.name}:")
            for s in self.sessions:
                print(f"- With {s['member']} on {s['date']} at {s['time']}")


# Example usage
if __name__ == "__main__":
    # Create members
    m1 = Member("M101", "Tehreem Fatima", "premium")
    m2 = Member("M102", "Tehreem Noor", "basic")

    # Create trainers
    t1 = Trainer("T201", "Trainer Tehreem", "Cardio")
    t2 = Trainer("T202", "Coach Tehreem", "Strength")

    # Mark attendance
    m1.mark_attendance("2025-10-14")
    m1.mark_attendance("2025-10-15")
    m2.mark_attendance("2025-10-14")
```

```
    # Mark attendance
    m1.mark_attendance("2025-10-14")
    m1.mark_attendance("2025-10-15")
    m2.mark_attendance("2025-10-14")

    # Schedule sessions
    t1.schedule_session(m1, "2025-10-16", "10:00 AM")
    t2.schedule_session(m2, "2025-10-17", "6:00 PM")

    # Display details
    print("\n--- Member Info ---")
    m1.display_info()
    m2.display_info()

    print("\n--- Trainer Info ---")
    t1.display_info()
    t2.display_info()

    print("\n--- Sessions ---")
    m1.display_sessions()
    t1.display_sessions()
```

Output

```
Attendance marked for Tehreem Fatima on 2025-10-14.
Attendance marked for Tehreem Fatima on 2025-10-15.
Attendance marked for Tehreem Noor on 2025-10-14.
Session scheduled: Tehreem Fatima with Trainer Trainer Tehreem on 2025-10-16 at 10:00 AM
Session scheduled: Tehreem Noor with Trainer Coach Tehreem on 2025-10-17 at 6:00 PM

--- Member Info ---
Member ID: M101
Name: Tehreem Fatima
Membership Type: premium
Total Attendance: 2 days
Member ID: M102
Name: Tehreem Noor
Membership Type: basic
Total Attendance: 1 days

--- Trainer Info ---
Trainer ID: T201
Name: Trainer Tehreem
Specialty: Cardio
Trainer ID: T202
Name: Coach Tehreem
Specialty: Strength

--- Sessions ---
Training Sessions for Tehreem Fatima:
- Trainer Tehreem on 2025-10-16 at 10:00 AM
Training Sessions for Trainer Tehreem:
- With Tehreem Fatima on 2025-10-16 at 10:00 AM
```

Question 15

```python
class BankAccount:
    def __init__(self, account_number, account_holder, balance=0.0):
        self.account_number = account_number
        self.account_holder = account_holder
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited: {amount}")
        else:
            print("Deposit amount must be positive!")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient balance!")
        elif amount <= 0:
            print("Withdrawal amount must be positive!")
        else:
            self.balance -= amount
            print(f"Withdrawn: {amount}")

    def display_balance(self):
        print(f"Account Holder: {self.account_holder}")
        print(f"Account Number: {self.account_number}")
        print(f"Current Balance: {self.balance}")

    def transfer(self, target_account, amount):
        if amount > self.balance:
            print("Transfer failed: Insufficient funds.")
        elif amount <= 0:
            print("Transfer amount must be positive.")
        else:
```

```
            self.balance -= amount
            target_account.balance += amount
            print(f"Transferred {amount} from {self.account_holder} to {target_account.account_holder}.")


class SavingsAccount(BankAccount):
    def __init__(self, account_number, account_holder, balance=0.0, interest_rate=0.03):
        super().__init__(account_number, account_holder, balance)
        self.interest_rate = interest_rate  # 3% default interest rate

    def add_interest(self):
        interest = self.balance * self.interest_rate
        self.balance += interest
        print(f"Interest added: {interest}")
        print(f"New Balance: {self.balance}")

    def withdraw(self, amount):
        # Savings accounts usually limit withdrawals
        if amount > self.balance:
            print("Insufficient balance in savings account!")
        elif amount > 50000:
            print("Withdrawal limit exceeded! (Max 50,000 per transaction)")
        else:
            self.balance -= amount
            print(f"Withdrawn {amount} from Savings Account.")


class CheckingAccount(BankAccount):
    def __init__(self, account_number, account_holder, balance=0.0, overdraft_limit=10000):
        super().__init__(account_number, account_holder, balance)
        self.overdraft_limit = overdraft_limit

    def withdraw(self, amount):
        # Checking accounts allow overdraft up to a limit
        if amount > self.balance + self.overdraft_limit:
```

```
if __name__ == "__main__":
    # Create accounts
    acc1 = SavingsAccount("SA001", "Tehreem Fatima", 100000)
    acc2 = CheckingAccount("CA002", "Tehreem Noor", 20000)

    # Display initial balances
    acc1.display_balance()
    acc2.display_balance()

    print("\n--- Transactions ---")
    acc1.deposit(5000)
    acc1.withdraw(60000)   # exceeds savings limit
    acc1.withdraw(40000)   # valid withdrawal
    acc1.add_interest()    # add interest

    acc2.withdraw(25000)   # overdraft used
    acc2.deposit(10000)

    print("\n--- Fund Transfer ---")
    acc1.transfer(acc2, 20000)

    # Display updated balances
    print("\n--- Final Balances ---")
    acc1.display_balance()
    acc2.display_balance()
```

Output

```
Account Holder: Tehreem Fatima
Account Number: SA001
Current Balance: 100000
Account Holder: Tehreem Noor
Account Number: CA002
Current Balance: 20000

--- Transactions ---
Deposited: 5000
Withdrawal limit exceeded! (Max 50,000 per transaction)
Withdrawn 40000 from Savings Account.
Interest added: 1950.0
New Balance: 66950.0
Withdrawn 25000 from Checking Account. Current balance: -5000
Deposited: 10000

--- Fund Transfer ---
Transferred 20000 from Tehreem Fatima to Tehreem Noor.

--- Final Balances ---
Account Holder: Tehreem Fatima
Account Number: SA001
Current Balance: 46950.0
Account Holder: Tehreem Noor
Account Number: CA002
Current Balance: 25000
```

Question 16

```
class Pizza:
    def __init__(self, size, crust_type, toppings=None):
        self.size = size
        self.crust_type = crust_type
        self.toppings = toppings if toppings else []

    def calculate_price(self):
        base_prices = {"small": 800, "medium": 1200, "large": 1600}
        crust_extra = {"thin": 0, "cheese_burst": 300, "stuffed": 200}
        topping_price = 100 * len(self.toppings)

        base = base_prices.get(self.size, 1000)
        crust = crust_extra.get(self.crust_type, 0)

        total = base + crust + topping_price
        return total

    def display_pizza(self):
        print(f"Pizza Size: {self.size.title()}")
        print(f"Crust Type: {self.crust_type.title()}")
        print("Toppings:", ", ".join(self.toppings) if self.toppings else "None")
        print(f"Total Price: Rs {self.calculate_price()}")


class PizzaOrderSystem:
    def __init__(self):
        self.orders = []

    def create_custom_pizza(self):
        print("Welcome to Tehreem's Pizza Corner!")
        size = input("Choose size (small/medium/large): ").lower()
        crust = input("Choose crust (thin/cheese_burst/stuffed): ").lower()
```

```
    toppings = []
    print("Enter toppings (type 'done' when finished):")
    while True:
        topping = input("> ").lower()
        if topping == "done":
            break
        toppings.append(topping)

    pizza = Pizza(size, crust, toppings)
    self.orders.append(pizza)
    print("\n--- Your Pizza Details ---")
    pizza.display_pizza()

def show_all_orders(self):
    print("\n--- All Orders ---")
    if not self.orders:
        print("No pizzas ordered yet.")
    else:
        for i, pizza in enumerate(self.orders, start=1):
            print(f"\nOrder #{i}")
            pizza.display_pizza()
```

```python
# Example usage
if __name__ == "__main__":
    system = PizzaOrderSystem()

    # Example: Create pizzas manually without user input
    p1 = Pizza("medium", "cheese_burst", ["mushroom", "olive", "jalapeno"])
    p2 = Pizza("large", "thin", ["pepperoni", "extra cheese"])

    system.orders.append(p1)
    system.orders.append(p2)

    print("--- Sample Orders ---")
    p1.display_pizza()
    p2.display_pizza()
```

Output

```
--- Sample Orders ---
Pizza Size: Medium
Crust Type: Cheese_Burst
Toppings: mushroom, olive, jalapeno
Total Price: Rs 1800
Pizza Size: Large
Crust Type: Thin
Toppings: pepperoni, extra cheese
Total Price: Rs 1800
```

Question 17

```python
class Car:
    def __init__(self, model, year, color, price):
        self.model = model
        self.year = year
        self.color = color
        self.price = price
        self.quality_checked = False

    def perform_quality_check(self):
        # Simple quality check logic (you can extend it)
        if self.price > 0 and self.year >= 2020 and self.color:
            self.quality_checked = True
            print(f"Quality Check Passed for {self.model} ({self.color}, {self.year})")
        else:
            print(f"Quality Check Failed for {self.model}")

    def display_info(self):
        print(f"Model: {self.model}")
        print(f"Year: {self.year}")
        print(f"Color: {self.color}")
        print(f"Price: Rs {self.price}")
        print(f"Quality Checked: {'Yes' if self.quality_checked else 'No'}")


class CarManufacturer:
    def __init__(self, name):
        self.name = name
        self.production_log = []
```

```python
    def produce_car(self, model, year, color, price):
        car = Car(model, year, color, price)
        car.perform_quality_check()
        self.production_log.append(car)
        print(f"Car Produced: {model} ({color}, {year}) by {self.name}")
        return car

    def display_production_log(self):
        print(f"\n--- {self.name} Production Log ---")
        if not self.production_log:
            print("No cars produced yet.")
        else:
            for i, car in enumerate(self.production_log, start=1):
                print(f"\nCar #{i}:")
                car.display_info()

    def production_summary(self):
        total_cars = len(self.production_log)
        passed = sum(1 for c in self.production_log if c.quality_checked)
        failed = total_cars - passed
        print(f"\n--- Production Summary for {self.name} ---")
        print(f"Total Cars Produced: {total_cars}")
        print(f"Passed Quality Check: {passed}")
        print(f"Failed Quality Check: {failed}")
```

```python
# Example usage
if __name__ == "__main__":
    manufacturer = CarManufacturer("Tehreem Motors")

    # Producing cars with different specs
    c1 = manufacturer.produce_car("Tehreem X1", 2025, "Red", 2800000)
    c2 = manufacturer.produce_car("Tehreem Swift", 2019, "Blue", 2200000)
    c3 = manufacturer.produce_car("Tehreem EV", 2024, "White", 3500000)

    # Show all cars and summary
    manufacturer.display_production_log()
    manufacturer.production_summary()
```

Output

```
Quality Check Passed for Tehreem X1 (Red, 2025)
Car Produced: Tehreem X1 (Red, 2025) by Tehreem Motors
Quality Check Failed for Tehreem Swift
Car Produced: Tehreem Swift (Blue, 2019) by Tehreem Motors
Quality Check Passed for Tehreem EV (White, 2024)
Car Produced: Tehreem EV (White, 2024) by Tehreem Motors

--- Tehreem Motors Production Log ---

Car #1:
Model: Tehreem X1
Year: 2025
Color: Red
Price: Rs 2800000
Quality Checked: Yes

Car #2:
Model: Tehreem Swift
Year: 2019
Color: Blue
Price: Rs 2200000
Quality Checked: No

Car #3:
Model: Tehreem EV
Year: 2024
Color: White
Price: Rs 3500000
Quality Checked: Yes

--- Production Summary for Tehreem Motors ---
Total Cars Produced: 3
Passed Quality Check: 2
Failed Quality Check: 1
```

Question 18

```
# ----- Student Class -----
class Student:
    def __init__(self, name, roll_no):
        self.name = name
        self.roll_no = roll_no
        self.attendance = 0
        self.assignments = {}  # {assignment_name: grade}

    def mark_attendance(self):
        self.attendance += 1

    def add_grade(self, assignment, grade):
        self.assignments[assignment] = grade

    def average_grade(self):
        if not self.assignments:
            return 0
        return sum(self.assignments.values()) / len(self.assignments)

    def display_info(self):
        print(f"Name: {self.name}")
        print(f"Roll No: {self.roll_no}")
        print(f"Attendance: {self.attendance} days")
        print("Grades:")
        for assignment, grade in self.assignments.items():
            print(f"  {assignment}: {grade}")
        print(f"Average Grade: {self.average_grade():.2f}")
        print("-" * 30)
```

```
# ----- Teacher Class -----
class Teacher:
    def __init__(self, name, subject):
        self.name = name
        self.subject = subject

    def assign_grade(self, student, assignment, grade):
        student.add_grade(assignment, grade)
        print(f"{self.name} assigned {grade} marks to {student.name} for {assignment}.")

    def mark_attendance(self, student):
        student.mark_attendance()
        print(f"Attendance marked for {student.name} by {self.name}.")


# ----- Classroom Class -----
class Classroom:
    def __init__(self, class_name, teacher):
        self.class_name = class_name
        self.teacher = teacher
        self.students = []

    def add_student(self, student):
        self.students.append(student)
        print(f"Student {student.name} added to class {self.class_name}.")

    def show_all_students(self):
        print(f"\n--- Students in {self.class_name} ---")
        for student in self.students:
            print(f"{student.roll_no} - {student.name}")
        print("-" * 30)

    def generate_report(self):
        print(f"\n=== {self.class_name} Class Report ===")
        print(f"Teacher: {self.teacher.name} ({self.teacher.subject})")
        print("----------------------------------")
```

```python
# ----- Example Usage -----
if __name__ == "__main__":
    # Create teacher and classroom
    teacher = Teacher("Ms. Tehreem", "Artificial Intelligence")
    classroom = Classroom("AI Class", teacher)

    # Create students
    s1 = Student("Ali", 1)
    s2 = Student("Sara", 2)
    s3 = Student("Bilal", 3)

    # Add students to class
    classroom.add_student(s1)
    classroom.add_student(s2)
    classroom.add_student(s3)

    # Mark attendance
    teacher.mark_attendance(s1)
    teacher.mark_attendance(s2)
    teacher.mark_attendance(s3)

    # Assign grades
    teacher.assign_grade(s1, "Assignment 1", 85)
    teacher.assign_grade(s2, "Assignment 1", 90)
    teacher.assign_grade(s3, "Assignment 1", 75)

    teacher.assign_grade(s1, "Assignment 2", 88)
    teacher.assign_grade(s2, "Assignment 2", 92)
    teacher.assign_grade(s3, "Assignment 2", 80)

    # Generate report
    classroom.generate_report()
```

Output

```
Student Ali added to class AI Class.
Student Sara added to class AI Class.
Student Bilal added to class AI Class.
Attendance marked for Ali by Ms. Tehreem.
Attendance marked for Sara by Ms. Tehreem.
Attendance marked for Bilal by Ms. Tehreem.
Ms. Tehreem assigned 85 marks to Ali for Assignment 1.
Ms. Tehreem assigned 90 marks to Sara for Assignment 1.
Ms. Tehreem assigned 75 marks to Bilal for Assignment 1.
Ms. Tehreem assigned 88 marks to Ali for Assignment 2.
Ms. Tehreem assigned 92 marks to Sara for Assignment 2.
Ms. Tehreem assigned 80 marks to Bilal for Assignment 2.

=== AI Class Class Report ===
Teacher: Ms. Tehreem (Artificial Intelligence)
----------------------------------
Name: Ali
Roll No: 1
Attendance: 1 days
Grades:
  Assignment 1: 85
  Assignment 2: 88
Average Grade: 86.50
```

Question 19

5]:
```python
# ----- Passenger Class -----
class Passenger:
    def __init__(self, name, passenger_id):
        self.name = name
        self.passenger_id = passenger_id

    def display_info(self):
        print(f"Passenger ID: {self.passenger_id}, Name: {self.name}")


# ----- Flight Class -----
class Flight:
    def __init__(self, flight_number, destination, departure_time, capacity):
        self.flight_number = flight_number
        self.destination = destination
        self.departure_time = departure_time
        self.capacity = capacity
        self.passengers = []  # list of Passenger objects

    def book_seat(self, passenger):
        if len(self.passengers) < self.capacity:
            self.passengers.append(passenger)
            print(f"Seat booked for {passenger.name} on Flight {self.flight_number}.")
        else:
            print(f"Sorry, Flight {self.flight_number} is fully booked.")

    def cancel_booking(self, passenger_id):
        for p in self.passengers:
            if p.passenger_id == passenger_id:
                self.passengers.remove(p)
                print(f"Booking canceled for {p.name} on Flight {self.flight_number}.")
                return
        print("Passenger not found in booking list.")
```

```
    def show_passenger_manifest(self):
        print(f"\n--- Passenger Manifest for Flight {self.flight_number} ---")
        if not self.passengers:
            print("No passengers booked yet.")
        else:
            for p in self.passengers:
                p.display_info()
        print(f"Total Passengers: {len(self.passengers)}/{self.capacity}")
        print("-" * 40)

    def display_flight_info(self):
        print(f"\nFlight Number: {self.flight_number}")
        print(f"Destination: {self.destination}")
        print(f"Departure Time: {self.departure_time}")
        print(f"Capacity: {self.capacity}")
        print(f"Seats Booked: {len(self.passengers)}")
        print("-" * 40)


# ----- Example Usage -----
if __name__ == "__main__":
    # Create a flight
    flight1 = Flight("PK305", "Dubai", "10:00 AM", 3)

    # Create passengers
    p1 = Passenger("Tehreem Fatima", "P001")
    p2 = Passenger("Tehreem Noor", "P002")
    p3 = Passenger("Tehreem Zahra", "P003")
    p4 = Passenger("Tehreem Aliya", "P004")  # extra passenger to test full booking

    # Display flight info
    flight1.display_flight_info()
```

```python
# Book seats
flight1.book_seat(p1)
flight1.book_seat(p2)
flight1.book_seat(p3)
flight1.book_seat(p4)  # should show full

# Show all passengers
flight1.show_passenger_manifest()

# Cancel a booking
flight1.cancel_booking("P002")

# Show updated manifest
flight1.show_passenger_manifest()
```

Output

```
Flight Number: PK305
Destination: Dubai
Departure Time: 10:00 AM
Capacity: 3
Seats Booked: 0
----------------------------------------
Seat booked for Tehreem Fatima on Flight PK305.
Seat booked for Tehreem Noor on Flight PK305.
Seat booked for Tehreem Zahra on Flight PK305.
Sorry, Flight PK305 is fully booked.

--- Passenger Manifest for Flight PK305 ---
Passenger ID: P001, Name: Tehreem Fatima
Passenger ID: P002, Name: Tehreem Noor
Passenger ID: P003, Name: Tehreem Zahra
Total Passengers: 3/3
----------------------------------------
Booking canceled for Tehreem Noor on Flight PK305.

--- Passenger Manifest for Flight PK305 ---
Passenger ID: P001, Name: Tehreem Fatima
Passenger ID: P003, Name: Tehreem Zahra
Total Passengers: 2/3
----------------------------------------
```

Question 20

```
]: # ----- Base Class -----
class SmartDevice:
    def __init__(self, name, status=False):
        self.name = name
        self.status = status  # True = ON, False = OFF

    def turn_on(self):
        self.status = True
        print(f"{self.name} is now ON.")

    def turn_off(self):
        self.status = False
        print(f"{self.name} is now OFF.")

    def show_status(self):
        state = "ON" if self.status else "OFF"
        print(f"{self.name} status: {state}")


# ----- SmartLight Class -----
class SmartLight(SmartDevice):
    def __init__(self, name, brightness=50):
        super().__init__(name)
        self.brightness = brightness

    def set_brightness(self, level):
        if 0 <= level <= 100:
            self.brightness = level
            print(f"{self.name} brightness set to {level}%.")
        else:
            print("Brightness level must be between 0 and 100.")
```

```
# ----- Thermostat Class -----
class Thermostat(SmartDevice):
    def __init__(self, name, temperature=22):
        super().__init__(name)
        self.temperature = temperature

    def set_temperature(self, temp):
        self.temperature = temp
        print(f"{self.name} temperature set to {temp}°C.")


# ----- SecurityCamera Class -----
class SecurityCamera(SmartDevice):
    def __init__(self, name, recording=False):
        super().__init__(name)
        self.recording = recording

    def start_recording(self):
        if not self.recording:
            self.recording = True
            print(f"{self.name} started recording.")
        else:
            print(f"{self.name} is already recording.")

    def stop_recording(self):
        if self.recording:
            self.recording = False
            print(f"{self.name} stopped recording.")
        else:
            print(f"{self.name} is not recording.")
```

```python
class SmartHome:
    def __init__(self, home_name):
        self.home_name = home_name
        self.devices = []

    def add_device(self, device):
        self.devices.append(device)
        print(f"{device.name} added to {self.home_name} system.")

    def show_all_devices(self):
        print(f"\n--- Devices in {self.home_name} ---")
        for device in self.devices:
            device.show_status()
        print("-" * 40)

    def turn_all_on(self):
        print("\nTurning ON all devices...")
        for device in self.devices:
            device.turn_on()

    def turn_all_off(self):
        print("\nTurning OFF all devices...")
        for device in self.devices:
            device.turn_off()

    def create_automation(self, routine_name):
        print(f"\n--- Running Automation: {routine_name} ---")
        if routine_name.lower() == "good morning":
            for device in self.devices:
                if isinstance(device, SmartLight):
                    device.turn_on()
                    device.set_brightness(80)
                elif isinstance(device, Thermostat):
                    device.turn_on()
                    device.set_temperature(24)
                elif isinstance(device, SecurityCamera):
                    device.stop_recording()
```

```python
        elif routine_name.lower() == "good night":
            for device in self.devices:
                if isinstance(device, SmartLight):
                    device.set_brightness(20)
                elif isinstance(device, Thermostat):
                    device.set_temperature(20)
                elif isinstance(device, SecurityCamera):
                    device.start_recording()
            self.turn_all_off()
            print("Good Night routine activated.")
        else:
            print("Unknown routine name.")


# ----- Example Usage -----
if __name__ == "__main__":
    # Create Smart Devices
    light1 = SmartLight("Bedroom Light")
    thermostat = Thermostat("Living Room Thermostat")
    camera = SecurityCamera("Main Door Camera")

    # Create Smart Home
    home = SmartHome("Tehreem Home")

    # Add devices
    home.add_device(light1)
    home.add_device(thermostat)
    home.add_device(camera)

    # Display all devices
    home.show_all_devices()

    # Run automation routines
    home.create_automation("Good Morning")
    home.show_all_devices()
```

Output

```
Bedroom Light added to Tehreem Home system.
Living Room Thermostat added to Tehreem Home system.
Main Door Camera added to Tehreem Home system.

--- Devices in Tehreem Home ---
Bedroom Light status: OFF
Living Room Thermostat status: OFF
Main Door Camera status: OFF
----------------------------------------

--- Running Automation: Good Morning ---
Bedroom Light is now ON.
Bedroom Light brightness set to 80%.
Living Room Thermostat is now ON.
Living Room Thermostat temperature set to 24°C.
Main Door Camera is not recording.
Good Morning routine activated.

--- Devices in Tehreem Home ---
Bedroom Light status: ON
Living Room Thermostat status: ON
Main Door Camera status: OFF
----------------------------------------

--- Running Automation: Good Night ---
Bedroom Light brightness set to 20%.
Living Room Thermostat temperature set to 20°C.
Main Door Camera started recording.

Turning OFF all devices...
Bedroom Light is now OFF.
Living Room Thermostat is now OFF.
Main Door Camera is now OFF.
Good Night routine activated.

--- Devices in Tehreem Home ---
Bedroom Light status: OFF
Living Room Thermostat status: OFF
Main Door Camera status: OFF
```

LAB - 【09】

What is NumPy (Numerical Python)?

NumPy is a **Python library** used for **numerical calculations**.
It is mainly used to work with:

- Arrays
- Matrices
- Mathematical operations

NumPy is **faster and more efficient** than normal Python lists.

Why NumPy is Important

- Handles large amounts of data
- Used in **Data Science, AI, and ML**
- Provides mathematical and statistical functions
- Base library for Pandas, Scikit-learn, TensorFlow

NumPy Array

A NumPy array is a **collection of same-type elements**.

Creating an Array

```
import numpy as np

arr = np.array([10, 20, 30, 40])
print(arr)
```

Difference: List vs NumPy Array

Python List	NumPy Array

Slow	Fast
Can store mixed data	Same data type
Less memory efficient	More memory efficient

Array Dimensions

- **1D** → list
- **2D** → matrix
- **3D** → tensor

```
a = np.array([1, 2, 3])          # 1D
b = np.array([[1, 2], [3, 4]])   # 2D
```

Array Properties

```
print(arr.ndim)   # Dimensions
print(arr.shape)  # Rows and columns
print(arr.size)   # Total elements
print(arr.dtype)  # Data type
```

Creating Special Arrays

```
np.zeros(5)       # All zeros
np.ones(4)        # All ones
np.arange(1, 10)  # Range
np.linspace(1, 5, 5)  # Equal values
```

Indexing & Slicing

```
arr = np.array([10, 20, 30, 40, 50])

print(arr[0])     # First element
print(arr[1:4])   # Slicing
```

Mathematical Operations

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(a + b)
print(a * b)
print(a ** 2)
```

Statistical Functions

```
data = np.array([10, 20, 30, 40])

print(np.mean(data))
print(np.sum(data))
print(np.max(data))
print(np.min(data))
print(np.std(data))
```

Reshaping Arrays

```
arr = np.array([1, 2, 3, 4, 5, 6])
new_arr = arr.reshape(2, 3)
print(new_arr)
```

Random Numbers

```
np.random.rand(3)       # Random floats
np.random.randint(1, 10, 5)  # Random integers
```

Lab Exercise

Question 1

[1]:
```
import numpy as np
```

[3]:
```
temp = np.array([21.1, 23.4, 19.8, 25.0, 26.5, 24.1, 22.0])
```

[13]:
```
temp_avg = np.mean(temp)
print(temp_avg)
```

```
23.12857142857143
```

[17]:
```
temp_max = np.max(temp)
print(temp_max)
```

```
26.5
```

[21]:
```
temp_min = np.min(temp)
print(temp_min)
```

```
19.8
```

[25]:
```
temp_diff = np.max(temp) - np.min(temp)
print (temp_diff)
```

```
6.699999999999999
```

[27]:
```
temp_diff = temp_max - temp_min
print (temp_diff)
```

```
6.699999999999999
```

Question 2

[39]:
```
daily_sales=np.array([
[12, 15, 14, 10, 9, 8, 7],
 [11, 12, 13, 9, 5, 8, 6],
 [7, 9, 12, 10, 11, 13, 12],
[10, 11, 12, 7, 8, 9, 11]])
print(daily_sales)
```

```
[[12 15 14 10  9  8  7]
 [11 12 13  9  5  8  6]
 [ 7  9 12 10 11 13 12]
 [10 11 12  7  8  9 11]]
```

[55]:
```
sum = np.sum(daily_sales, axis=1)
print(sum)
```

```
[75 64 74 68]
```

[57]:
```
highest = np.argmax(daily_sales)
print(highest)
```

```
1
```

[61]:
```
most_sales = np.argmax(daily_sales)
print(most_sales)
```

```
1
```

Question 3

4]:
```
marks = np.array([56, 78, 45, 90, 88, 76, 59, 62])
```

8]:
```
new_marks = marks + 5
print(new_marks)
```

```
[61 83 50 95 93 81 64 67]
```

2]:
```
stds = np.sum (new_marks > 60)
print(stds)
```

```
7
```

8]:
```
median = np.median(new_marks)
sd = np.std(new_marks)
print("median:", median, "\nstandard deviation:", sd)
```

```
median: 74.0
standard deviation: 15.105876340020794
```

Question 5

```
heart_rates = np.random.randint(60, 150, (10, 4))

print("Heart rate readings:\n", heart_rates)
```

```
Heart rate readings:
 [[ 81 102 119 138]
 [101 106 105 148]
 [ 86 113 107  75]
 [127 117  75 114]
 [129 114  79 149]
 [113 114 113  64]
 [110  86  79 139]
 [120 148 115 116]
 [ 68 112  89  96]
 [143  81  87  70]]
```

```
daily_avg = heart_rates.mean(axis=1)
print(daily_avg)
```

```
[110.   115.    95.25 108.25 117.75 101.   103.5  124.75  91.25  95.25]
```

```
abnormal_readings = heart_rates > 120
print(abnormal_readings)
```

```
[[False False False  True]
 [False False False  True]
 [False False False False]
 [ True False False False]
 [ True False False  True]
 [False False False False]
 [False False False  True]
 [False  True False False]
 [False False False False]
 [ True False False False]]
```

```
max = heart_rates.max()
print(max)
```

```
149
```

```
min = heart_rates.min()
print(min)
```

```
64
```

Question 6

[57]:
```
branch_A = np.array([122, 135, 150, 160, 155, 140])
branch_B = np.array([110, 130, 148, 165, 158, 145])
print("branch A", branch_A, "\nbrach B", branch_B)
```

```
branch A [122 135 150 160 155 140]
brach B [110 130 148 165 158 145]
```

[65]:
```
difference = branch_A - branch_B
print(difference)
```

```
[12  5  2 -5 -3 -5]
```

[67]:
```
avg_A = branch_A.mean()
avg_B = branch_B.mean()

print(avg_A)
print(avg_B)
```

```
143.66666666666666
142.66666666666666
```

[69]:
```
if avg_A > avg_B:
    print("Branch A performed better overall")
elif avg_B > avg_A:
    print("Branch B performed better overall")
else:
    print("Both branches performed equally")
```

```
Branch A performed better overall
```

[73]:
```
correlation = np.corrcoef(branch_A, branch_B)[0, 1]
print(correlation)
```

```
0.9811677353198398
```

Question 7

[4]:
```
import numpy as np
img = np.array([
    [100, 120, 130, 90],
    [80, 110, 140, 100],
    [70, 90, 120, 130],
    [110, 140, 150, 160]
])

print(img)
```

```
[[100 120 130  90]
 [ 80 110 140 100]
 [ 70  90 120 130]
 [110 140 150 160]]
```

```
normalized = img / 255
print(normalized)
```

```
[[0.39215686 0.47058824 0.50980392 0.35294118]
 [0.31372549 0.43137255 0.54901961 0.39215686]
 [0.2745098  0.35294118 0.47058824 0.50980392]
 [0.43137255 0.54901961 0.58823529 0.62745098]]
```

[12]:
```
transformed = img * 1.2
print(transformed)
```

```
[[120. 144. 156. 108.]
 [ 96. 132. 168. 120.]
 [ 84. 108. 144. 156.]
 [132. 168. 180. 192.]]
```

Question 8 ¶

]:
```
data = np.random.randint(1, 101, size=(100, 4))
print("Original Dataset:\n", data)
```

```
Original Dataset:
 [[ 13  76  50  33]
 [ 89  51  68  41]
 [  7  11   6  38]
 [ 24  50   8  92]
 [ 70   2  70  14]
 [ 50  85  53  67]
 [ 72  72  75  86]
 [ 32   1  72  29]
 [ 60   6  82   3]
 [ 16  69  92  64]
 [ 11  49  20  74]
```

```
mean = data.mean(axis=0)
std = data.std(axis=0)

x_scaled = (data - mean) / std
print(x_scaled)
```

```
[[-1.13943626e+00  7.20986239e-01 -1.72840898e-01 -5.69268392e-01]
 [ 1.33406147e+00 -3.73080805e-02  4.23162198e-01 -2.70243629e-01]
 [-1.33471240e+00 -1.25057899e+00 -1.62973735e+00 -3.82377915e-01]
 [-7.81430008e-01 -6.76398533e-02 -1.56351479e+00  1.63603923e+00]
 [ 7.15687042e-01 -1.52356495e+00  4.89384764e-01 -1.27945220e+00]
 [ 6.47665854e-02  9.93972193e-01 -7.35070485e-02  7.01586850e-01]
```

```
print(x_scaled.mean(axis=0))
```

```
[4.66293670e-17 9.54791801e-17 5.32907052e-17 7.88258347e-17]
```

```
print( x_scaled.var(axis=0))
```

```
[1. 1. 1. 1.]
```

```
print("NumPy Mean Check:", np.mean(x_scaled, axis=0))
print("NumPy Variance Check:", np.var(x_scaled, axis=0))
```

```
NumPy Mean Check: [4.66293670e-17 9.54791801e-17 5.32907052e-17 7.88258347e-17]
NumPy Variance Check: [1. 1. 1. 1.]
```

Question 9

```
traffic = np.random.randint(0, 101, size=(24, 6))
print( traffic)
```

```
[[ 72  67  77  91  22   9]
 [  5  26   7  65  14  33]
 [  7  59   0  68  80  49]
 [ 23  89  83  64  45  51]
 [ 79  48  58  54  29 100]
```

```
hourly_avg = traffic.mean(axis=1)
print(hourly_avg)
```

```
[56.33333333 25.         43.83333333 59.16666667 61.33333333 54.
 76.66666667 41.         58.5        38.83333333 63.         30.16666667
 62.66666667 63.5        30.5        52.66666667 69.66666667 41.33333333
 55.5        36.         64.83333333 50.         53.83333333 24.        ]
```

```
sensor_totals = traffic.sum(axis=0)
busiest_sensor = np.argmax(sensor_totals)
print(sensor_totals)
print( busiest_sensor)
```

```
[1258 1158 1321 1253 1091 1193]
2
```

```
traffic_capped = np.clip(traffic, 0, 50)
print( traffic_capped)
```

```
[[50 50 50 50 22  9]
 [ 5 26  7 50 14 33]
 [ 7 50  0 50 50 49]
 [23 50 50 50 45 50]
 [50 48 50 50 29 50]
 [50 22 24 50 50 50]]
```

```
energy_usage = np.sum(traffic_capped * 0.5)
print("Total Energy Usage:", energy_usage)
```

```
Total Energy Usage: 2730.0
```

Question 10

15]:
```
F = np.array([
    [5, 2, 1],
    [3, 1, 4],
    [6, 3, 2],
    [4, 2, 5],
    [7, 1, 3]
], dtype=float)
print (F)
```

```
[[5. 2. 1.]
 [3. 1. 4.]
 [6. 3. 2.]
 [4. 2. 5.]
 [7. 1. 3.]]
```

17]:
```
T = np.array([
    [2, 1, 0],
    [1, 3, 1],
    [0, 2, 2]
], dtype=float)
print(T)
```

```
[[2. 1. 0.]
 [1. 3. 1.]
 [0. 2. 2.]]
```

19]:
```
F_new = F.dot(T)
print( F_new)
```

```
[[12. 13.  4.]
 [ 7. 14.  9.]
 [15. 19.  7.]
 [10. 20. 12.]
 [15. 16.  7.]]
```

```
magnitudes = np.linalg.norm(F_new, axis=1)
print( magnitudes)
```

```
[18.13835715 18.05547009 25.19920634 25.37715508 23.02172887]
```

```
max_index = np.argmax(magnitudes)
print(f"Component {max_index + 1}")
print(f"Highest magnitude: {magnitudes[max_index]:.4f}")
```

```
Component 4
Highest magnitude: 25.3772
```

```
eigvals, eigvecs = np.linalg.eig(T)
print("\nEigenvalues:\n", eigvals)
print("\nEigenvectors (columns):\n", eigvecs)
```

```
Eigenvalues:
 [4.30277564 2.         0.69722436]

Eigenvectors (columns):
 [[-3.11546502e-01  7.07106781e-01  3.86413754e-01]
 [-7.17421694e-01  2.70737023e-17 -5.03410424e-01]
 [-6.23093003e-01 -7.07106781e-01  7.72827507e-01]]
```

LAB - 【11】

Learning Objectives :

- To understand the **TensorFlow library**, its purpose in machine learning and deep learning, and its key features such as multi-platform support, scalability, and visualization with TensorBoard.
- To learn the concept of **tensors, operations, and computational graphs**, and how TensorFlow executes computations using eager execution or sessions.

What is TensorFlow?

TensorFlow is an **open-source library developed by Google** for machine learning (ML) and deep learning (DL). It is widely used to build and train neural networks for tasks like image recognition, natural language processing, and predictive modeling.

Key Features of TensorFlow:

- **Multi-platform:** Runs on CPU, GPU, mobile devices, and even browsers.
- **Flexible:** Supports high-level APIs like Keras for easy model building.
- **Scalable:** Can handle large datasets and distributed training.
- **Visualization:** Comes with TensorBoard for monitoring and visualizing model performance.

Why is it called TensorFlow?

- **Tensor** → a fancy name for multi-dimensional data (like arrays, matrices).
- **Flow** → means data flows through different layers of a neural network.

So **TensorFlow = data (tensors) flowing through operations**.

Where is TensorFlow used?

TensorFlow is used in:

- Image recognition
- Face detection
- Voice assistants
- Text classification

- Chatbots
- Medical analysis
- Recommendation systems (like Netflix)
- Self-driving cars

```python
import tensorflow as tf
from tensorflow import keras

# Simple model with one hidden layer
model = keras.Sequential([
    keras.layers.Dense(10, activation='relu'),
    keras.layers.Dense(1)
])

# Compile the model
model.compile(optimizer='adam', loss='mse')

# Dummy data
x = [1, 2, 3, 4]
y = [2, 4, 6, 8]

# Train the model
model.fit(x, y, epochs=50)

# Predict
print(model.predict([5]))
```

2. How TensorFlow Works

TensorFlow works with **tensors**, which are just **multi-dimensional arrays**. It builds a **computation graph**, where each node represents a mathematical operation, and edges represent data (tensors) flowing between operations.

Key Components:

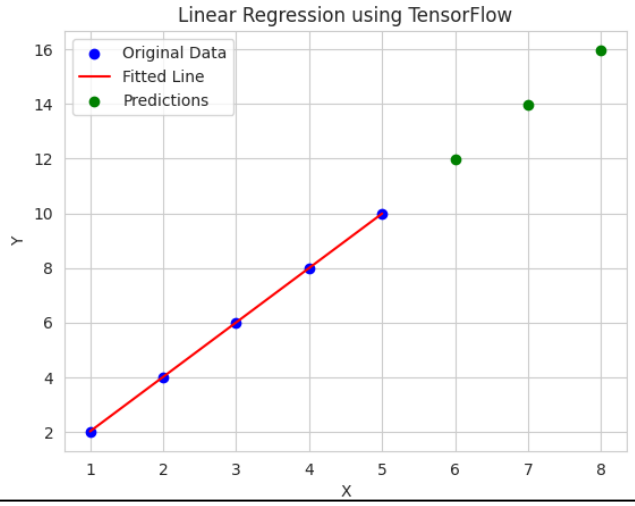
1. **Tensors:** Basic data structure (like arrays or matrices).
   Example: Scalars (0D), Vectors (1D), Matrices (2D), etc.
2. **Operations (Ops):** Mathematical operations applied to tensors (like addition, multiplication).
3. **Computational Graph:** A graph representing the flow of data through operations.
4. **Sessions (TF1.x) / Eager Execution (TF2.x):**
   - TensorFlow 1.x used sessions to run graphs.
   - TensorFlow 2.x executes operations immediately (like regular Python), making it easier to debug

Implementation

```python
# Import necessary libraries
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Prepare Data
# Features (X) and labels (Y)
X = np.array([1, 2, 3, 4, 5], dtype=float)
Y = np.array([2, 4, 6, 8, 10], dtype=float)  # Linear relationship: y = 2*x

# Step 2: Build the Model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(units=1, input_shape=[1])  # 1 input, 1 output
])

# Step 3: Compile the Model
model.compile(
    optimizer='sgd',               # Stochastic Gradient Descent optimizer
    loss='mean_squared_error'      # Loss function
)

# Step 4: Train the Model
history = model.fit(X, Y, epochs=1000, verbose=0)  # Train silently

# Step 5: Make Predictions
new_X = np.array([6, 7, 8], dtype=float)
predictions = model.predict(new_X)
print("Predictions for [6, 7, 8]:", predictions.flatten())

# Step 6: Visualize the Results
plt.scatter(X, Y, color='blue', label='Original Data')  # Original points
plt.plot(X, model.predict(X), color='red', label='Fitted Line')  # Regression line
plt.scatter(new_X, predictions, color='green', label='Predictions')  # Predictions
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Linear Regression using TensorFlow")
plt.legend()
plt.show()
```

Output

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 56ms/step
Predictions for [6, 7, 8]: [11.986539  13.9809065 15.975274 ]
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 53ms/step
```



Lab Exercise

Question 1

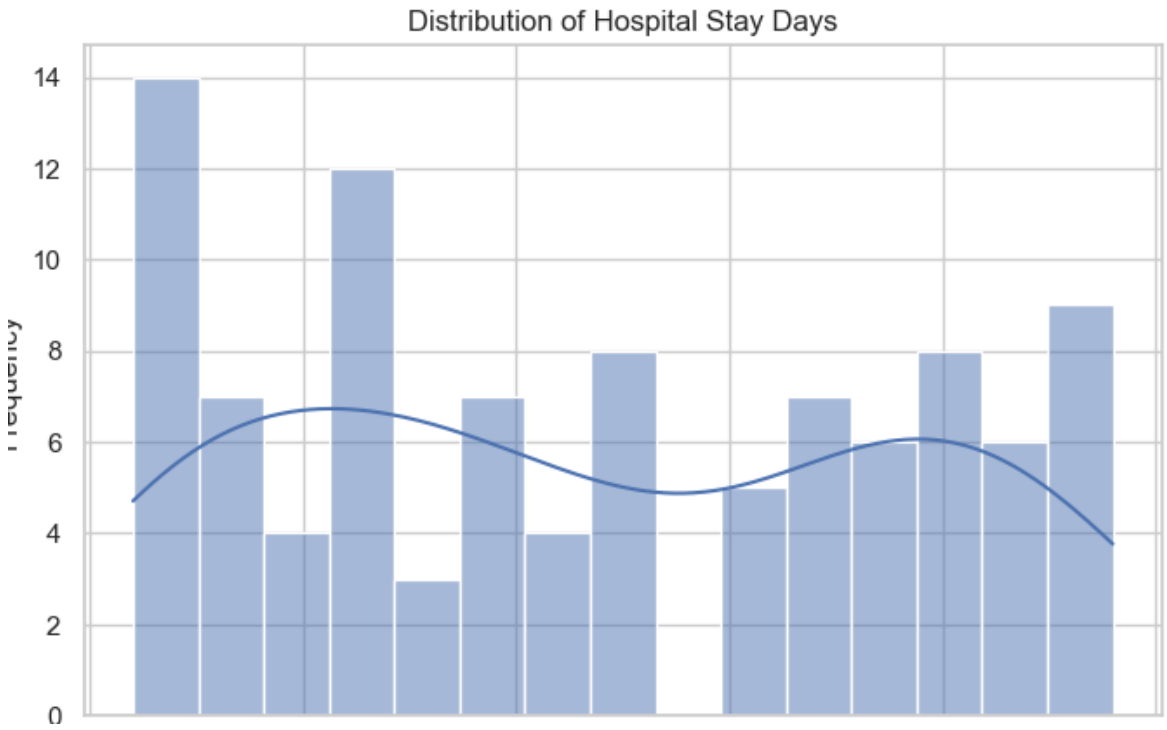
In [2]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

sns.set(style="whitegrid")
```

In [3]:
```
np.random.seed(42)

data = {
    "age": np.random.randint(20, 80, 100),
    "disease_type": np.random.choice(
        ["Cardiac", "Orthopedic", "Neurology", "Respiratory"], 100
    ),
    "hospital_stay_days": np.random.randint(1, 25, 100)
}

df = pd.DataFrame(data)
df.head()
```

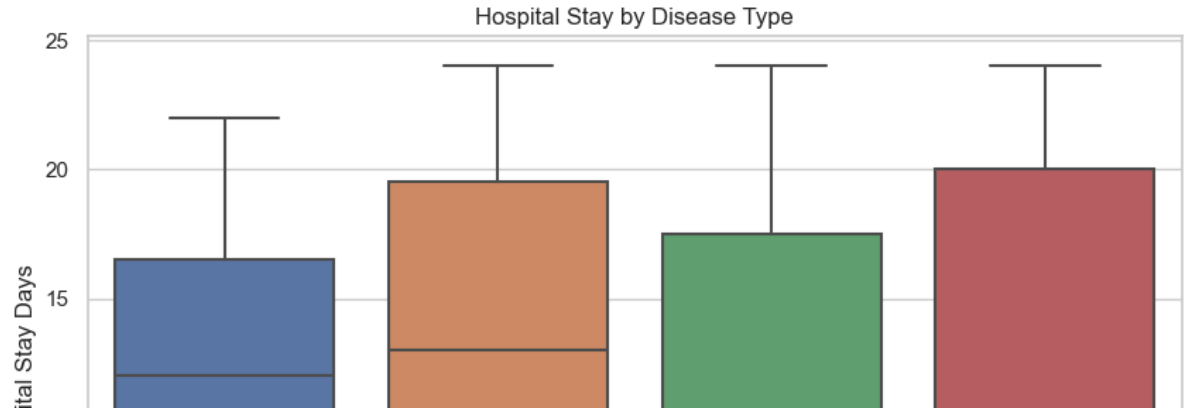
Out[3]:

	age	disease_type	hospital_stay_days
0	58	Orthopedic	15
1	71	Neurology	11
2	48	Respiratory	4
3	34	Cardiac	13
4	62	Orthopedic	7

In [4]:
```
plt.figure(figsize=(8, 5))
sns.histplot(df["hospital_stay_days"], bins=15, kde=True)
plt.title("Distribution of Hospital Stay Days")
plt.xlabel("Hospital Stay Days")
plt.ylabel("Frequency")
plt.show()
```



```
plt.figure(figsize=(10, 6))
sns.boxplot(x="disease_type", y="hospital_stay_days", data=df)
plt.title("Hospital Stay by Disease Type")
plt.xlabel("Disease Type")
plt.ylabel("Hospital Stay Days")
plt.show()
```




```
]: median_stay = df.groupby("disease_type")["hospital_stay_days"].median()

highest_median_disease = median_stay.idxmax()

print("Median Hospital Stay (Days) by Disease Type:")
print(median_stay)

print(f"\nDisease with the highest median stay: {highest_median_disease}")
```

```
Median Hospital Stay (Days) by Disease Type:
disease_type
Cardiac          9.5
Neurology       13.0
Orthopedic      12.0
Respiratory      9.0
Name: hospital_stay_days, dtype: float64

Disease with the highest median stay: Neurology
```

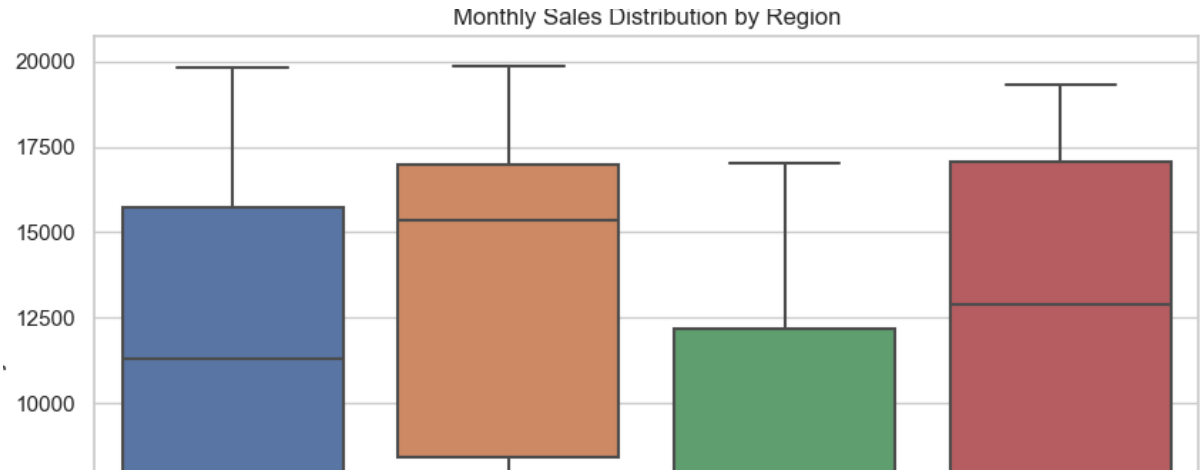
Question 2

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

sns.set(style="whitegrid")
```

```
np.random.seed(0)

data = {
    "region": np.random.choice(
        ["North", "South", "East", "West"], 120
    ),
    "monthly_sales": np.random.randint(2000, 20000, 120)
}

df = pd.DataFrame(data)
df.head()
```

	region	monthly_sales
0	North	17127
1	West	12959
2	South	4957
3	North	6469
4	West	16165

```
plt.figure(figsize=(10, 6))
sns.boxplot(x="region", y="monthly_sales", data=df)
plt.title("Monthly Sales Distribution by Region")
plt.xlabel("Region")
plt.ylabel("Monthly Sales")
plt.show()
```



```
sales_variability = df.groupby("region")["monthly_sales"].std()

print("Sales Variability (Standard Deviation) by Region:")
print(sales_variability)
```

```
Sales Variability (Standard Deviation) by Region:
region
East     5352.080694
North    5677.867290
South    4704.985318
West     4598.970624
Name: monthly_sales, dtype: float64
```

Question 3

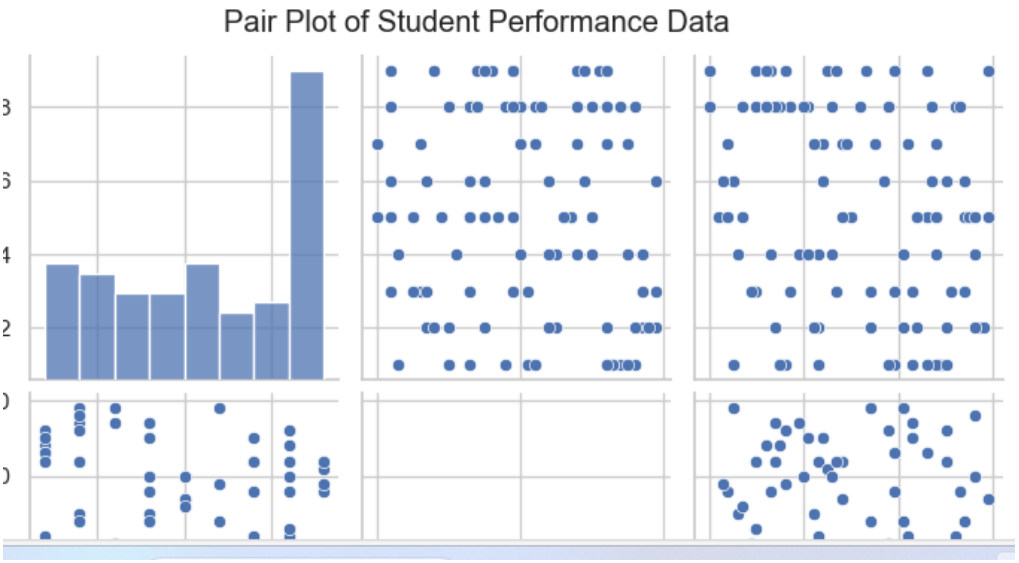
```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

sns.set(style="whitegrid")
```

n [12]:
```
np.random.seed(1)

data = {
    "study_hours": np.random.randint(1, 10, 100),
    "attendance": np.random.randint(60, 100, 100),
    "final_score": np.random.randint(40, 100, 100)
}

df = pd.DataFrame(data)
df.head()
```

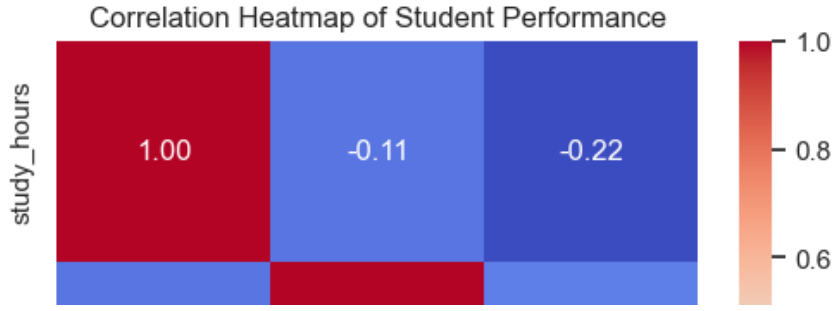
ut[12]:

	study_hours	attendance	final_score
0	6	62	87
1	9	88	79

```
sns.pairplot(df)
plt.suptitle("Pair Plot of Student Performance Data", y=1.02)
plt.show()
```




```
plt.figure(figsize=(6, 5))
correlation = df.corr()

sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Heatmap of Student Performance")
plt.show()
```




```
correlation["final_score"].sort_values(ascending=False)
```

```
final_score    1.000000
attendance    -0.080921
study_hours   -0.224538
Name: final_score, dtype: float64
```

Question 4

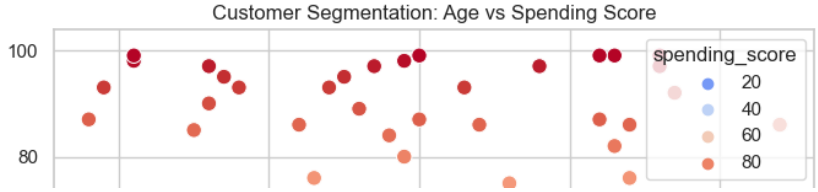
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

sns.set(style="whitegrid")
```

```
np.random.seed(42)

data = {
    "age": np.random.randint(18, 65, 150),
    "annual_income": np.random.randint(15000, 120000, 150),
    "spending_score": np.random.randint(1, 100, 150)
}

df = pd.DataFrame(data)
df.head()
```

	age	annual_income	spending_score
0	56	33141	24

```
plt.figure(figsize=(8, 6))
sns.scatterplot(
    data=df,
    x="annual_income",
    y="spending_score",
    hue="spending_score",
    palette="viridis",
    s=80
)

plt.title("Customer Segmentation: Income vs Spending Score")
plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.show()
```


```
plt.figure(figsize=(8, 6))
sns.scatterplot(
    data=df,
    x="age",
    y="spending_score",
    hue="spending_score",
    palette="coolwarm",
    s=80
)

plt.title("Customer Segmentation: Age vs Spending Score")
plt.xlabel("Age")
plt.ylabel("Spending Score")
plt.show()
```

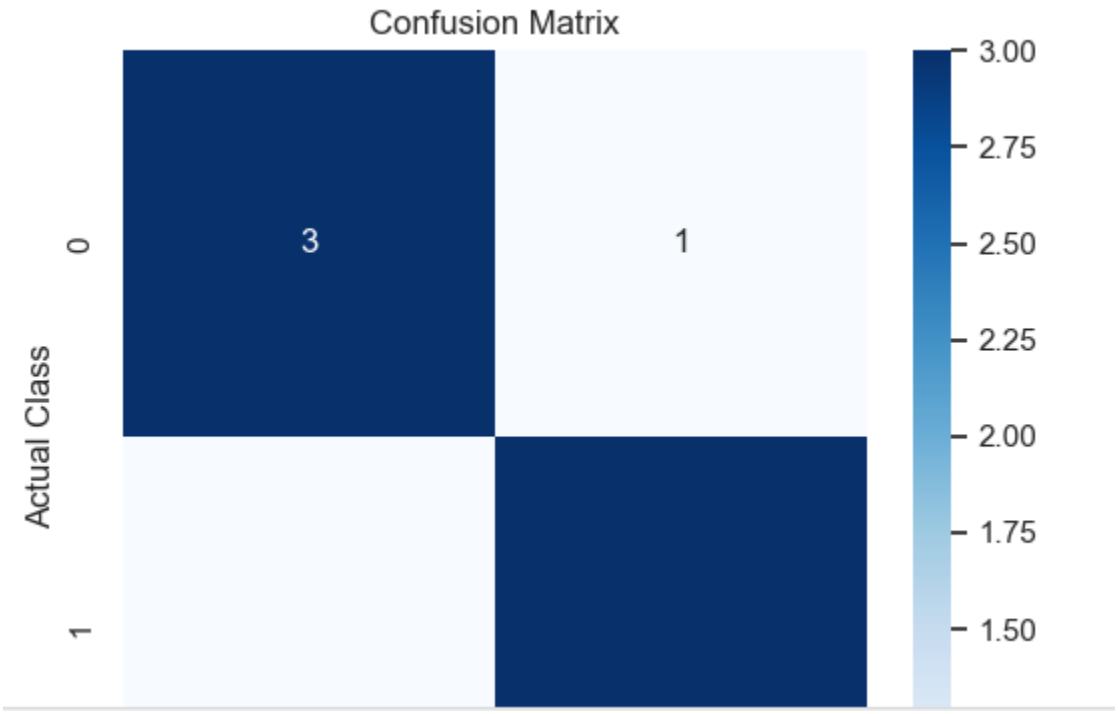


Question 5

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
```

```
y_true = [0, 1, 0, 1, 1, 0, 1, 0]
y_pred = [0, 1, 0, 0, 1, 0, 1, 1]

cm = confusion_matrix(y_true, y_pred)
```

```
plt.figure(figsize=(6, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    cbar=True
)

plt.title("Confusion Matrix")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.show()
```



Question 6

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

# Sample dataset
data = pd.DataFrame({
    "tenure": [1, 12, 24, 36, 6, 48],
    "monthly_charges": [70, 80, 60, 90, 75, 85],
    "contract_type": ["Month-to-month", "One year", "Two year", "One year", "Month-to-month", "Two year"],
    "churn": [1, 0, 0, 0, 1, 0]
})

X = data.drop("churn", axis=1)
y = data["churn"]

# Encode categorical variable
preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(), ["contract_type"])
    ],
    remainder="passthrough"
)

X_encoded = preprocessor.fit_transform(X)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X_encoded, y, test_size=0.3, random_state=42)
```

```
# Logistic Regression
model = LogisticRegression()
model.fit(X_train, y_train)

# Evaluation
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

Output

```
Accuracy: 0.5
Confusion Matrix:
 [[0 1]
 [0 1]]
```

Question 7

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Sample dataset
data = pd.DataFrame({
    "area": [800, 1000, 1200, 1500, 1800],
    "bedrooms": [2, 2, 3, 3, 4],
    "location_score": [6, 7, 8, 9, 9],
    "price": [5000000, 6500000, 7500000, 9000000, 11000000]
})

X = data.drop("price", axis=1)
y = data["price"]

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)

# Linear Regression
model = LinearRegression()
model.fit(X_train, y_train)

# Prediction
predictions = model.predict(X_test)

# Evaluation
mse = mean_squared_error(y_test, predictions)
print("Mean Squared Error:", mse)
```

Output

```
Mean Squared Error: 109814388020.13188
```

Question 8

```python
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import precision_score, recall_score, f1_score

# Sample dataset
data = pd.DataFrame({
    "glucose": [120, 140, 130, 150, 160],
    "BMI": [25, 30, 28, 35, 33],
    "age": [25, 45, 35, 50, 60],
    "outcome": [0, 1, 0, 1, 1]
})

X = data.drop("outcome", axis=1)
y = data["outcome"]

# Normalize data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3, random_state=2)

# Decision Tree
model = DecisionTreeClassifier()
model.fit(X_train, y_train)
```

```python
# Evaluation
y_pred = model.predict(X_test)
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))
```

Output

```
Precision: 0.5
Recall: 1.0
F1 Score: 0.6666666666666666
```

Question 9

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

# Email dataset
emails = [
    "Win a free lottery now",
    "Meeting scheduled tomorrow",
    "Claim your free prize",
    "Project deadline reminder"
]

labels = [1, 0, 1, 0]  # 1 = Spam, 0 = Not Spam

# TF-IDF
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(emails)

# Naive Bayes
model = MultinomialNB()
model.fit(X, labels)

# Test new email
new_email = ["Free money waiting for you"]
new_email_vec = vectorizer.transform(new_email)

prediction = model.predict(new_email_vec)
print("Spam Prediction:", "Spam" if prediction[0] == 1 else "Not Spam")
```

Output

```
Spam Prediction: Spam
```

Question 10

```python
import pandas as pd
from sklearn.ensemble import RandomForestClassifier

# Sample dataset
data = pd.DataFrame({
    "attendance": [60, 85, 70, 90, 50],
    "mid_marks": [40, 70, 55, 80, 35],
    "final_marks": [45, 75, 60, 85, 40],
    "at_risk": [1, 0, 1, 0, 1]
})

X = data.drop("at_risk", axis=1)
y = data["at_risk"]

# Random Forest
model = RandomForestClassifier(n_estimators=100, random_state=3)
model.fit(X, y)

# Prediction
prediction = model.predict([[65, 50, 55]])
print("At Risk Prediction:", "At Risk" if prediction[0] == 1 else "Not At Risk")

# Feature Importance
print("\nFeature Importance:")
for feature, importance in zip(X.columns, model.feature_importances_):
    print(feature, ":", importance)
```

Output

```
At Risk Prediction: At Risk

Feature Importance:
attendance : 0.3473684210526316
mid_marks : 0.35789473684210527
final_marks : 0.29473684210526313
```